

Enhancing Performance and Quality of Context Retrieval in Retrieval-Augmented Generation (RAG) Systems

Paschalidis Marios

Thesis submitted for the degree of
Master of Artificial Intelligence

Supervisor

Academic – Dr. Tim Van de Cruys

Assessors

Deloitte – Yuanyuan Li

Deloitte – Florian Geron

Deloitte – Grzegorz Przesniak

Assistant-supervisors

Deloitte – Simon Sels

© Publication year KU Leuven – Faculty of Engineering Science

Published by List of authors on one line,

Faculty of Engineering Science, Kasteelpark Arenberg 1 bus 2200, B-3001 Leuven

All rights reserved. No part of the publication may be reproduced in any form by print, photoprint, microfilm, electronic or any other means without written permission from the publisher.

This publication contains the study work of a student in the context of the academic training and assessment. After this assessment no correction of the study work took place.

Preface

Completing this thesis has been the most demanding and the most instructive part of my studies in Artificial Intelligence, and it would not have taken its present form without the people who supported me along the way. I am especially grateful to Grzegorz Przesniak, whose guidance and sustained interest accompanied every stage of this work. I would also like to thank Yuanyuan Li and Florian Geron, who consistently pushed me to deepen my analysis and to raise the quality of what I produced. My thanks extend to the entire Deloitte team for welcoming me so openly and for giving me the chance to see how artificial intelligence is applied to real business problems. I am equally grateful to my academic supervisor, Dr. Tim Van de Cruys, for his valuable input and for the perspective he brought to the project.

Finally, beyond their professional guidance, I want to thank the people mentioned above, both at Deloitte and at the university, for their care and understanding during a difficult time. They gave me the room and the time I needed, and I am deeply grateful for it.

Marios Paschalidis

Contents

Preface.....	1
Contents.....	2
Abstract.....	5
List of figures and tables.....	6
List of abbreviations and symbols.....	8
1 Introduction	11
1.1 Motivation	11
1.2 Problem statement.....	13
1.3 Research questions	13
1.4 Contributions	14
1.5 Scope and context.....	14
1.6 Thesis outline	15
2 Background and Related Work	16
2.1 Retrieval-augmented generation and pipeline architectures.....	16
2.2 Retrieval paradigms	17
2.3 Agentic retrieval.....	18
2.4 Legal information retrieval and BSARD	19
2.5 Structure-aware retrieval	19
2.6 Autonomous, reference-free evaluation	20
2.7 Research gap.....	21
3 Datasets and Experimental Setup	22
3.1 The BSARD canonical corpus (RQ1 and RQ3)	22
3.2 The PDF-extracted corpus (RQ2).....	23
3.2.1 Corpus construction and ground-truth drift.....	24
3.3 Query stratification and the 48-question subset.....	25
3.4 Evaluation metrics	25
3.5 Reproducibility and tooling.....	26

4	Methodology	27
4.1	RQ1 — Retrieval method comparison	27
4.1.1	Tier 1 — Sparse retrieval.....	27
4.1.2	Tier 2 — Dense retrieval.....	28
4.1.3	Tier 3 — Hybrid fusion	28
4.1.4	Tier 4 — LLM-Judge and agentic retrieval.....	29
4.2	RQ2 — Structure-aware retrieval	30
4.2.1	Arm 1 — Naive hybrid baseline	31
4.2.2	Arm 2A — Metadata-enriched retrieval.....	31
4.2.3	Arm 2B — PageIndex navigation.....	31
4.2.4	Arm 2C — Agentic deep-tree navigation (follow-up)	32
4.2.5	The common scoring path	32
4.3	RQ3 — Autonomous evaluator panel.....	33
4.3.1	The evaluators.....	33
4.3.2	The gold-ceiling diagnostic	33
4.3.3	Validation methodology	34
5	Results	35
5.1	RQ1 — Retrieval method comparison	35
5.1.1	Tier 1 — Sparse retrieval.....	35
5.1.2	Tier 2 — Dense retrieval.....	36
5.1.3	Tier 3 — Hybrid fusion	37
5.1.4	Tier 4 — LLM-Judge and agentic retrieval.....	38
5.1.5	Cross-tier synthesis and the matched-pool agentic comparison	40
5.2	RQ2 — Structure-aware retrieval	42
5.2.1	Cross-arm headline — structure helps ranking, not coverage.....	42
5.2.2	Arm 2A — where the lift comes from, and what drives difficulty	43
5.2.3	Arm 2B — why PageIndex collapses.....	44
5.2.4	Arm 2C — rescuing navigation with a deeper tree	45
5.3	RQ3 — Autonomous evaluator comparison.....	48
5.3.1	System-level rank agreement.....	48

5.3.2	Where agreement breaks down.....	49
5.3.3	Trustworthiness and the gold-ceiling inversion	50
6	Discussion	52
6.1	Answer to RQ1.....	52
6.2	Answer to RQ2.....	52
6.3	Answer to RQ3.....	53
6.4	Cross-cutting implications for legal-RAG system design	53
6.5	Limitations and generalisation	54
7	Conclusion and Future Work	56
7.1	Summary of contributions.....	56
7.2	Future work.....	57
7.3	Closing reflection.....	57
	Bibliography	58
	Code of Conduct	61

Abstract

Retrieval-augmented generation (RAG) pairs a retriever with a generative language model, and in specialised domains the retriever, not the generator, usually limits answer quality: a passage that is never retrieved cannot be used. This follows from how generative language models are built: they are trained on a fixed, historical snapshot of data, so where that snapshot does not adequately cover a specialised domain, the model cannot be relied on to generate accurate or comprehensive answers from its parameters alone, and retrieving the relevant domain-specific text becomes essential. Regulatory law makes this particularly hard, since statutory text is dense and formal and the questions users ask seldom match its wording. This thesis examines how to retrieve more effectively over such corpora through three experiments on the Belgian Statutory Article Retrieval Dataset (BSARD), each varying a single design choice and considering both what it gains and what it costs.

This topic and the regulatory domain were chosen for a concrete, applied reason: the work was carried out in collaboration with Deloitte, which has developed an internal accelerator for regulatory processes such as gap analysis and requirements extraction. Retrieval is a key component of that accelerator, so a better understanding of how retrieval can be optimised for dense, formally-written regulatory text directly supports improving the underlying solution.

The first experiment compares the standard retrieval methods: sparse lexical search, dense vector search, a hybrid of the two, and a language model used first as a re-ranker and then as an agentic controller. Most of the improvement comes from adding the language model as a scorer, while the heavier agentic loops built on top of it add little. The second investigates enriching the index with document structure metadata and using this structure at query time to identify the most relevant sections of a document, and finds structure most useful in the index. The third considers evaluation without ground truth: reference-free evaluators can substitute for supervised metrics only in part, and they differ considerably in how far they can be trusted.

Across the three studies, simpler choices placed at the right point in the pipeline tend to outperform more elaborate alternatives, and the thesis turns this into concrete recommendations for retrieval in regulatory RAG systems.

List of figures and tables

List of figures

Figure 1 — Dense retrieval: stratified Recall@10 by encoder	37
Figure 2 — Tier 3 hybrid hyperparameter sweeps (RRF, linear, SGDR).....	38
Figure 3 — T4.0 LLM-Judge: Recall@k by depth and stratified Recall@10	39
Figure 4 — T4.1 CRAG: Recall@k by depth and stratified Recall@10	39
Figure 5 — T4.2 ReAct: Recall@k by depth and stratified Recall@10	40
Figure 6 — Recall@k across the four-tier retrieval ladder.....	40
Figure 7 — Stratified Recall@10 and the two difficulty axes, canonical systems	41
Figure 8 — Recall@k across the structure-aware arms.....	43
Figure 9 — Arm 2A: the six metadata variants	43
Figure 10 — Per-document recall and Recall@10 by answer cardinality, by arm	44
Figure 11 — Arm 2B: navigated-vs-padded recall and the wrong-chapter miss mechanism	45
Figure 12 — Per-document Recall@10 across all five arms	45
Figure 13 — Arm 2C: reach-vs-selection (padding removed) and gold-article fate.....	46
Figure 14 — Code Civil: the CNR lever ladder	47
Figure 15 — Per-document Recall@10: simple navigation versus CNR, with the metadata ceiling	48
Figure 16 — Why Arm 2C wins on some documents and not others	48
Figure 17 — System-level ranking agreement (19 retrievers).....	49
Figure 18 — Regime-dependence of evaluator agreement	50
Figure 19 — Gold-ceiling diagnostic and headroom by question difficulty	51

List of tables

Table 1 — Key statistics of the canonical BSARD corpus	23
Table 2 — The curated five-document RQ2 set	24
Table 3 — Sparse retrieval: representative configurations.....	36
Table 4 — Dense retrieval: representative zero-shot configurations	36
Table 5 — Hybrid fusion: best configuration per family.....	37
Table 6 — Tier 4 canonical results.....	38
Table 7 — The RQ1 retrieval ladder (best system per tier)	40
Table 8 — Agentic versus non-agentic comparison.....	41
Table 9 — Cross-arm headline metrics (725 questions)	42
Table 10 — Arm 2A within-arm: enrichment and granularity.....	44

Table 11 — Arm 2C component ablation (Code Civil)	47
Table 12 — System-level rank agreement and document-level agreement.....	49
Table 13 — Gold-ceiling diagnostic	50

List of abbreviations and symbols

Abbreviations

Azure DI	Azure Document Intelligence
BERT	Bidirectional Encoder Representations from Transformers
BGE-M3	BAAI General Embedding (multilingual)
BM25	Best Match 25 (probabilistic relevance ranking function)
BSARD	Belgian Statutory Article Retrieval Dataset
CI	Confidence Interval
CNR	Corrective Navigation and Re-ranking
CRAG	Corrective Retrieval-Augmented Generation
FAISS	Facebook AI Similarity Search
FTS5	Full-Text Search v5 (SQLite)
HyDE	Hypothetical Document Embeddings
IR	Information Retrieval
LLM	Large Language Model
MAP	Mean Average Precision
mE5	multilingual E5
MRR	Mean Reciprocal Rank
NDCG	Normalised Discounted Cumulative Gain
NLP	Natural Language Processing
R@k	Recall at k
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
RRF	Reciprocal Rank Fusion
SGDR	Sparse-Gated Dense Re-ranking (this thesis)
TF-IDF	Term Frequency–Inverse Document Frequency
UMBRELA / eRAG / RAGAS	reference-free evaluators

Symbols

k	retrieval depth / rank cut-off
ρ	Spearman rank correlation
τ	Kendall rank correlation
κ	Cohen's κ
α	linear-interpolation weight
p	significance-test p-value
n	number of items

1 Introduction

1.1 Motivation

A large language model stores a great deal of knowledge in its parameters. That knowledge is fixed at training time, hard to inspect, and apt to be supplemented by confident fabrication. Retrieval-Augmented Generation (RAG) addresses these weaknesses by coupling the model with a retrieval step over an external corpus, so that what it generates is grounded in documents that can be cited, updated, and audited. The retriever then becomes the decisive component. A generator can only reason over the passages it is handed, so retrieval quality sets a hard ceiling on the quality of the final answer.

This dependence is most acute in regulatory domains. Statutory text is terminologically dense, heavily cross-referential, and written in a formal register that ordinary users' questions rarely match word for word. Someone asking whether a landlord may keep a deposit will seldom phrase the question in the vocabulary of the civil code that answers it. Retrieval, not generation, is therefore the rate-limiting step in a legal RAG system, and the distance between how a question is asked and how the law is written is the central obstacle this thesis confronts.

The work was carried out in collaboration with Deloitte, where the motivation is concrete. Specifically, Deloitte has built an internal accelerator that applies RAG to regulatory workflows such as gap analysis, comparing a client's policies against applicable regulation, and requirements extraction, deriving actionable obligations from regulatory text. In both use cases, the quality of what is retrieved directly determines the reliability of the accelerator's output, which is what makes the retrieval question practically, not just academically, important. Organisations increasingly want to deploy RAG over large bodies of regulation, and two obstacles stand in the way, each bound up with the other. The first is that it is unclear which retrieval strategy actually performs well on dense legal text, or whether the more elaborate, agent-driven pipelines that have lately become popular are worth their added cost. Cost, here and throughout the thesis, refers to the amount of language-model work a method invokes per query, the number of LLM calls or reasoning steps, not to wall-clock time. GPU-dependent experiments were run on Azure cloud

compute instances, where wall-clock time depends on factors beyond the language-model call itself, such as shared infrastructure and instance provisioning, so it is not used as a reliable basis for comparison (§5.1.5); the number of language-model calls a pipeline makes is used instead as the practical measure of what it costs to run. The second is that retrieval quality is hard to measure once a system is deployed, where the expert relevance annotations of academic benchmarks do not exist. How to retrieve well, and how to know that you have retrieved well: these two questions motivate the three research questions of this thesis.

1.2 Problem statement

The core difficulty here is a vocabulary gap between the way citizens ask questions and the way the law is written. On the Belgian Statutory Article Retrieval Dataset (BSARD; described fully in §2.4), the benchmark used throughout this thesis, the median lexical (Jaccard) overlap between a question and the article that answers it is just 0.045.¹ A question and its ground-truth article, in other words, typically share almost no exact vocabulary. Exact-match retrieval is structurally insufficient as a result: any system that leans on surface term overlap will, by construction, miss most of the relevant articles. This one observation motivates the progression the thesis investigates, from lexical to semantic and finally to language-model-driven retrieval.

A second property compounds the difficulty. Most BSARD questions are answered by several articles rather than one: 65.5 % require more than a single relevant article, with a mean of 6.18 per question. What matters is therefore how many of the relevant articles a system recovers, not whether it surfaces one good one, and this makes recall rather than top-rank precision the primary concern. Semantic paraphrasing and multi-article ground truth together define the failure modes against which every method in this thesis is measured. They also expose three open problems: which retrieval strategy best copes with the vocabulary gap, whether the structure of legal documents can be exploited to retrieve better, and how retrieval quality can be measured when expert annotations are unavailable.

1.3 Research questions

This thesis is organised around a single primary research question, addressed through three more specific sub-questions.

Primary research question. *How can the performance and quality of context retrieval in retrieval-augmented generation be enhanced for regulatory document corpora, and which retrieval strategies, structural approaches, and evaluation methods best support this goal?*

¹ Jaccard overlap measures how much vocabulary two texts share, as a proportion of their combined unique words, ranging from 0 (no shared words) to 1 (identical wording); the median reports the typical value across all question–article pairs. A value of 0.045 means a typical question and its ground-truth article share almost none of their wording. This reflects a real gap in how the two are written: questions are phrased by everyday citizens in ordinary language, without legal terminology, while articles are drafted by legal experts in precise, formal regulatory language. The lower this value, the harder it is for retrieval methods that rely on matching exact words to succeed.

RQ1 — Retrieval method comparison. *How do lexical, dense, hybrid, and agentic retrieval approaches compare in their ability to retrieve relevant context from regulatory corpora, and under what query characteristics and document conditions does each approach excel or fail?*

RQ2 — Structure-aware retrieval. *Does exploiting the hierarchical and relational structure of regulatory documents at the indexing stage through chunking and metadata enrichment, and at the retrieval stage through hierarchical navigation improve retrieval quality, and under what conditions and at what cost?*

RQ3 — Autonomous evaluation. *Can an autonomous, reference-free evaluation pipeline reliably assess and rank retrieval outputs in a regulatory domain without ground-truth annotations, and how well does its ranking agree with supervised metrics when ground truth is available as a validation signal?*

1.4 Contributions

This thesis makes the following contributions.

(i) A systematic four-tier comparison of sparse, dense, hybrid, and agentic retrieval on a closed regulatory statutory corpus, designed to separate the contribution of introducing a language model into the retrieval pipeline from that of the agentic machinery built around it.

(ii) A head-to-head comparison of three structure-aware retrieval approaches through a single common scoring path, examining whether and where exploiting document structure helps retrieval at indexing time versus at retrieval time, together with a deep-tree follow-up that probes whether the limits of hierarchical navigation are ones of coverage or of ranking.

(iii) A validation of reference-free retrieval evaluation against supervised metrics and a gold-only oracle, assessing which reference-free signals can stand in for supervised quality when ranking retrieval systems, and where their absolute scores can be trusted.

1.5 Scope and context

The thesis follows a two-phase design. All controlled experiments are run on the canonical BSARD text (Phase 1), while application to a PDF-extracted corpus from Justel (the source of BSARD's statutory articles, but here in its current, up-to-date form) is reserved for the structure-aware study and for testing generalisation. Several

boundaries are deliberate. The dense-retrieval comparison is *zero-shot*: no encoder is fine-tuned on the benchmark, so what it measures is retrieval architecture rather than task-specific training. The thesis also evaluates *retrieved context directly* rather than generated answers. In the fourth retrieval tier the language model acts as a relevance scorer or agentic reasoner over retrieved articles, never as an answer generator, so no answer-faithfulness or hallucination metrics are computed. All experiments use the French BSARD; the Dutch variant is out of scope. The work was carried out in collaboration with Deloitte, which shaped its applied orientation and supplied the document-processing tooling used to build the structure-aware corpus. This tooling is an existing PDF processing pipeline that uses Azure Document Intelligence together with a series of post-processing steps to extract, clean, and improve the content of regulatory documents, recreating their document structure and producing a logical chunking of the text. This preprocessing pipeline is treated as a fixed input and is out of scope for this thesis, which focuses on the retrieval component built on top of it.

1.6 Thesis outline

The remainder of the thesis proceeds as follows. Chapter 2 surveys the state of the art across retrieval-augmented generation, the retrieval paradigms, agentic retrieval, legal information retrieval, structure-aware retrieval, and reference-free evaluation, then closes by stating the research gap. Chapter 3 establishes the datasets, stratification, metrics, and infrastructure shared by the three studies. Chapter 4 sets out each study's methodology and Chapter 5 reports the results. Chapter 6 discusses the findings, their implications for legal-RAG system design, and their limitations; Chapter 7 concludes and identifies future work.

2 Background and Related Work

2.1 Retrieval-augmented generation and pipeline architectures

Retrieval-augmented generation was introduced by [Lewis et al. \(2020\)](#), a framework that couples a parametric language model with a non-parametric retrieval component over an external corpus. It belongs to a broader line of retrieval-augmented language models from around the same period, exemplified by REALM ([Guu et al., 2020](#)). These established that conditioning generation on retrieved text makes a model's knowledge updatable without retraining, attributable to a source, and auditable. As instruction-tuned large language models rose to prominence, the pattern became the dominant way of grounding them in domain-specific or proprietary collections, and it now supports a substantial literature surveyed by [Gao et al. \(2023b\)](#). It is the setting for all three research questions of this thesis.

Work extending the framework has advanced along two largely independent axes, both directly relevant here. The first is **retrieval quality**: *which documents* are placed before the model, running from sparse lexical matching through dense semantic retrieval to hybrid fusion and learned re-rankers (§2.2). The second is **pipeline architecture**: *how* retrieval and generation are organised. On this axis the literature has moved from the original single-pass *retrieve-then-read* design ([Lewis et al., 2020](#)) to iterative schemes that interleave retrieval with reasoning (IRCoT, [Trivedi et al., 2023](#)), then to self-reflective variants in which the model decides when and what to retrieve and critiques its own output (Self-RAG; [Asai et al., 2024](#)), and finally to corrective schemes that add an explicit retrieval-quality check with a recovery path when retrieval fails (CRAG; [Yan et al., 2024](#)).

These two axes organise the thesis directly. The retrieval-quality axis is the subject of the first three tiers of the RQ1 comparison (sparse, dense, hybrid), while the pipeline-architecture axis motivates the fourth, in which a language model enters first as a relevance scorer and then as the controller of corrective and reasoning loops. Framing the field this way sharpens the central RQ1 question. More elaborate pipeline architectures are plainly more *complex*; whether that complexity buys better *retrieval* is the open empirical question, and if it does, whether the gain comes from the presence of the language model or from the agentic machinery built around it.

2.2 Retrieval paradigms

Sparse lexical retrieval. The classical approach scores a query against a document by their term overlap, weighted by term and inverse-document frequency. BM25 and its variants, formalised within the probabilistic relevance framework ([Robertson & Zaragoza, 2009](#)), remain strong and computationally cheap baselines, well suited to settings where exact terminology matters, as it often does in legal text. Their structural weakness is the *vocabulary gap*: because they match surface forms, they fail when a query and a document express the same concept in different words. Learned sparse retrievers narrow this gap by predicting term expansions while keeping an efficient inverted index, but it remains the defining limitation of lexical matching, and §1.2 identified it as the dominant difficulty on the benchmark used here.

Dense retrieval. Dense bi-encoders embed queries and documents independently into a shared vector space and rank by inner product, capturing semantic similarity that exact term overlap misses (dense passage retrieval, [Karpukhin et al., 2020](#)). The central practical distinction is between *fine-tuned* encoders, trained on in-domain query–document pairs, and *zero-shot* use of a pretrained encoder. Generalising zero-shot across domains is not straightforward: the BEIR benchmark ([Thakur et al., 2021](#)) showed that retrievers strong in-domain can transfer poorly. This motivated contrastively pretrained encoders usable without fine-tuning (Contriever, [Izacard et al., 2022](#)) and, more recently, multilingual retrieval encoders (multilingual E5, [Wang et al., 2024](#); BGE-M3, [Chen et al., 2024a](#)) and decoder-only large language models adapted to produce embeddings, now routinely compared on broad embedding benchmarks (MTEB, [Muennighoff et al., 2023](#)). This thesis works in the zero-shot setting throughout, to isolate retrieval architecture from task-specific training (§1.5).

Hybrid retrieval. Since sparse and dense retrieval fail on different queries, combining them is a natural step. Two families dominate. *Score interpolation* takes a weighted sum of the two systems' normalised scores; *rank fusion* combines them by the reciprocals of the ranks they assign rather than their raw scores, its canonical instance being Reciprocal Rank Fusion ([Cormack et al., 2009](#)). The choice of fusion function measurably affects retrieval quality ([Bruch et al., 2023](#)), and rank fusion is attractive precisely because it sidesteps the score-calibration problem: sparse and dense scores live on incomparable scales. It proves the strongest mechanism in the RQ1 hybrid tier.

Learned re-ranking. A complementary line re-orders an initial candidate pool with a more expensive but more accurate model: cross-encoders that jointly encode the query and each candidate, late-interaction models that retain token-level representations (ColBERT, [Khattab & Zaharia, 2020](#)), and, increasingly, large

language models prompted to order candidates directly (RankGPT, [Sun et al., 2023](#)). This last development is conceptually where the RQ1 language-model scorer sits: it swaps a conventional cross-encoder for a prompted language model. The boundary between a learned re-ranker and an "agentic" retrieval step is exactly what the fourth RQ1 tier is designed to probe.

2.3 Agentic retrieval

The pipeline-architecture axis of §2.1 culminates in *agentic* approaches, in which a language model does not merely read retrieved text but actively controls the retrieval process: judging relevance, deciding whether to retrieve again, and reformulating its own queries. Three primitives from this literature are reused directly in the fourth tier of the RQ1 comparison, layered so that each adds one mechanism over the last.

The language model as a relevance judge. A capable language model can be prompted to assess the relevance or quality of a piece of text, an idea formalised as *LLM-as-a-judge* ([Zheng et al., 2023](#)) and shown to correlate with human judgement on generation-quality tasks ([Liu et al., 2023](#)). Applied to retrieval, the model scores each (query, document) pair and the candidate pool is re-ordered by that score, making it a drop-in replacement for the conventional cross-encoder re-ranker of §2.2. This is a *non-agentic* use: one pass, no loop, no actions.

Corrective retrieval. CRAG ([Yan et al., 2024](#)) wraps any retriever in a correction loop: a lightweight evaluator inspects the retrieved set and, if it is judged insufficient, triggers a recovery path. In the original design that path is a fallback to web search, and more generally query reformulation, a technique studied in its own right for retrieval-augmented models ([Ma et al., 2023](#)). A closed statutory corpus has no web to fall back on, so this thesis re-casts the recovery path as intra-corpus query rewriting (the closed-domain adaptation set out in §4.1.4).

Reasoning-and-acting agents. ReAct ([Yao et al., 2023](#)) interleaves chain-of-thought reasoning with tool-invoking actions in a Thought–Action–Observation loop, part of a broader line giving language models the ability to call external tools. A related family has the model reflect on and critique its own retrieval and generation (Self-RAG; [Asai et al., 2024](#)). These approaches are powerful, but markedly more expensive than a single scoring pass, and that cost frames the empirical question the thesis pursues: does the agentic mechanism itself improve retrieval, or is the benefit attributed to it really just the benefit of having a capable language model in the loop at all?

2.4 Legal information retrieval and BSARD

Legal information retrieval has developed its own benchmarks and models, most of them for English or supranational EU law. The landscape spans case-law retrieval (the COLIEE competition; [Goebel et al., 2024](#)) and large-scale classification of EU legislation (EUR-Lex; [Chalkidis et al., 2019](#)), supported by domain-adapted language models such as LEGAL-BERT ([Chalkidis et al., 2020](#)). One thing this body of work largely lacks is a retrieval benchmark over *national civil-law statutes* in a language other than English, annotated for relevance at the level of individual articles.

The Belgian Statutory Article Retrieval Dataset (BSARD; [Louis & Spanakis, 2022](#)) fills exactly this gap. It is, to date, the only public retrieval benchmark for French Belgian statutory law with expert article-level relevance judgements: French legal questions, each annotated with the statutory articles that answer it, over a corpus of civil-law articles drawn from the official consolidated legislation. It has since supported further retrieval-augmented legal question-answering research ([Louis et al., 2024](#)), and it is the controlled setting for RQ1 and RQ3 of this thesis.

Two properties of BSARD, both introduced as motivation in §1.2, drive the method comparison throughout this work. The **vocabulary gap**, the very low median lexical overlap between a question and its ground-truth article, makes lexical matching alone structurally insufficient, which motivates the dense and hybrid tiers. **Multi-article ground truth**, most questions need several articles to be answered fully, makes recall the primary concern and caps the achievable recall at low cut-offs for high-multiplicity questions. Together these define the failure-mode strata against which every method is later analysed (Chapter 3).

2.5 Structure-aware retrieval

The retrieval methods of §2.2 operate over a corpus that has been flattened into independent passages, discarding the layout and hierarchy of the documents those passages came from. Regulatory documents are deeply structured, though: a statute is organised into books, titles, chapters, and numbered articles, and an article's meaning and findability depend on where it sits in that hierarchy. One line of work asks whether this structure can be exploited to retrieve better, at two stages: when the corpus is *indexed*, and when it is *searched*.

Structure at indexing time. The unit a corpus is chunked into materially affects retrieval: fixed-size windows that cut across semantic boundaries retrieve dirtier units than boundary-respecting chunks, and the granularity of the indexed unit is itself a design variable with measurable effects ([Chen et al., 2024b](#)). Cleaner units are

only part of it. Each structural unit can also be *enriched* before indexing with metadata describing its place in the hierarchy (its book, chapter, and section) and its content (an LLM-generated summary or salient legal terms), giving a semantic retriever more signal to match a question against. This is the first hypothesis RQ2 tests.

Structure at retrieval time. A stronger use treats the hierarchy as a navigable index. Tree-organised retrieval methods such as RAPTOR ([Sarathi et al., 2024](#)) build a tree of recursive summaries and retrieve over it. The most aggressive form abandons the vector index altogether and has a language model descend the document's table of contents, book to chapter to article, the way a jurist narrows a search; PageIndex-style navigation exemplifies this [software footnote: Vectify AI PageIndex]. Exploiting structure this way first means recovering it from the document's layout, a document-AI problem in its own right, addressed for this work in Chapter 3.

These two stages, and the increasing structural commitment they represent, map directly onto the three approaches compared in RQ2: a structure-agnostic baseline, index-time metadata enrichment, and vectorless hierarchical navigation, together with a follow-up that rebuilds the navigation tree to a fuller depth (§4.2 and §5.2).

2.6 Autonomous, reference-free evaluation

Every metric in §2.2–§2.5 presupposes ground-truth relevance judgements (qrels). A benchmark such as BSARD provides them, but the arbitrary corpora over which retrieval is deployed in practice do not, and producing expert annotations there is slow and costly. Reference-free evaluation sets out to assess and rank retrieval systems *without* consulting ground truth, typically by prompting a strong language model to act as the judge.

Most off-the-shelf tools in this space judge **answer** quality rather than retrieval quality. RAGAS ([Es et al., 2024](#)) scores the faithfulness and answer relevance of a generated answer given its context, and G-Eval ([Liu et al., 2023](#)) applies LLM rubrics to generated text. Each of these needs a generated answer, and often a reference answer, so it evaluates the *generation* stage rather than the retriever in isolation.

A smaller set of evaluators targets **retrieval** directly. UMBRELA ([Upadhyay et al., 2024](#)) prompts a language model to assign a graded relevance label to each (query, document) pair, in the TREC relevance-assessment tradition; eRAG ([Salemi & Zamani, 2024](#)) measures each retrieved document's utility through a downstream answerability test. Context-precision metrics can also be adapted to a retrieval-only setting by replacing the missing reference answer with a hypothetical one generated from the query, repurposing the HyDE technique originally introduced for zero-shot

dense retrieval ([Gao et al., 2023a](#)). RQ3 assembles a panel of these retrieval-oriented, reference-free evaluators and asks what the literature has largely left open for a regulatory domain: when ground truth happens to be available, does the reference-free ranking actually agree with the supervised one, and where can its scores be trusted?

2.7 Research gap

The preceding survey locates three open problems that this thesis addresses, one per research question.

A controlled comparison of retrieval paradigms, isolating the agentic mechanism (RQ1). The literature offers each rung of the retrieval ladder — sparse, dense, hybrid (§2.2), and a progression of increasingly agentic pipelines (§2.3) — yet rarely compares them end-to-end under one protocol on a single closed corpus, and almost never separates what the *agentic mechanism* contributes from what is gained simply by introducing a language model into the loop. Whether corrective and reasoning loops earn their substantial additional cost over a single LLM scoring pass, at a matched candidate-pool size, remains unsettled.

Structure-aware retrieval, compared head-to-head (RQ2). Indexing-time enrichment and retrieval-time hierarchical navigation (§2.5) have each been studied on their own, but they are seldom placed side by side against a common structure-agnostic baseline, through a single scoring path, on the same documents and ground truth. So three things remain open: how much document structure helps, which way of exploiting it helps, and at what cost.

Reference-free evaluation of retrieval, validated against ground truth (RQ3). Most autonomous evaluators judge answer quality rather than retrieval quality (§2.6), and the retrieval-oriented ones are rarely validated against supervised metrics in a regulatory domain where ground truth happens to exist. Whether a reference-free panel can reliably *rank* retrieval systems, and where its scores can be trusted, is still open.

3 Datasets and Experimental Setup

This chapter establishes the experimental ground shared by the three studies: the corpora they retrieve over, the way questions are stratified for analysis, the metrics used to score retrieval, and the infrastructure that makes the results reproducible. Two evaluation settings are used, and they must be kept distinct: a canonical text corpus for RQ1 and RQ3 (§3.1), and a structured, PDF-extracted corpus for RQ2 (§3.2). Per-method details such as hyperparameter grids, prompts, and model configurations are deferred to Chapter 4.

RQ2 is conducted on a structured, PDF-extracted corpus, described in full in §3.2. Unlike the canonical corpus, which is a flat set of articles, this setting preserves each document's hierarchy — book, title, chapter, section, and article — recovered from its source PDF, and it is this hierarchy that the structure-aware methods exploit. From the 49 extracted documents, five were selected for the study against three criteria: a satisfactory proportion of articles that exactly match their BSARD counterparts, so that ground truth is reliably locatable in the extracted text; a satisfactory number of annotated questions per document; and a satisfactory variety of question types, so that the comparison is not dominated by a single retrieval regime. The resulting five-document set carries 725 questions in total (Table 2).

3.1 The BSARD canonical corpus (RQ1 and RQ3)

RQ1 and RQ3 are conducted on the canonical Belgian Statutory Article Retrieval Dataset (BSARD; [Louis & Spanakis, 2022](#)), introduced in §2.4. Loaded from the public release, the corpus contains 22,633 statutory articles, which the benchmark pairs with 1,108 expert-annotated French legal questions, each linked to the one or more articles that answer it. BSARD provides an official train/test split, and all RQ1 and RQ3 results in this thesis are reported on its 222-question test split. Evaluating on this split keeps the comparison aligned with the original BSARD paper and supplies a published benchmark against which the methods here can be measured.

Statistic	Value
Articles	22,633
Questions (total)	1,108
— training split	886
— test split (all final RQ1/RQ3 results)	222
Median question–article Jaccard overlap	0.045
Single-article questions	382 (34.5 %)
Multi-article questions	726 (65.5 %)

Statistic	Value
Mean relevant articles per question	6.18

Table 1 — Key statistics of the canonical BSARD corpus

Two of these statistics define the comparison setting for every method, and both were previewed as motivation in §1.2. The median question–article Jaccard overlap of 0.045 quantifies the vocabulary gap: a question and its ground-truth article typically share almost no exact terms, so lexical matching alone is structurally limited. With 65.5 % of questions requiring more than one article (mean 6.18 each), the appropriate measure of success is recall rather than the rank of a single best hit, and the achievable recall at small cut-offs is capped for high-multiplicity questions. The training split is used for tuning and model selection, as described per method in Chapter 4.

3.2 The PDF-extracted corpus (RQ2)

RQ2 cannot be studied on the canonical corpus, because the document structure it depends on — books, titles, chapters, sections, and the table of contents that organises them — has been stripped away to leave a flat set of articles. RQ2 therefore operates on whole regulatory documents parsed from their official source PDFs with their hierarchy intact, and retrieval is performed *per document*: each approach searches within a single code's own article set, and each question is scored against that code's ground truth. This per-document setting is also what makes hierarchical navigation meaningful, since a navigation tree is built for one document at a time.

Structural extraction. The structure-aware arms of RQ2 operate over a structural decomposition of each document: its articles and their place in a Book → Title → Chapter → Section → Article hierarchy, recovered from the source PDFs with Azure Document Intelligence, provided through the Deloitte collaboration (§1.5). This recovered hierarchy is the substrate that the metadata-enriched and navigation arms exploit (§4.2).

The curated five-document set. Working from extracted PDFs introduces a hazard absent from the canonical text: the published ground truth can drift away from the document on disk as statutory articles are amended, renumbered, or abrogated over time. An audit of candidate documents found one whose ground-truth articles had been almost entirely replaced by abrogation stubs in the source PDF, leaving its ground truth unlocatable, so it was excluded. The study therefore runs on a curated set of five clean documents, 725 questions in all, chosen after confirming that the bulk of each document's ground-truth articles are present in the extracted text.

Document	Questions
Code Civil	252
Code Judiciaire (larger part)	204
Code du Logement	133
Code Judiciaire (smaller part)	71
Code Pénal	65
Total	725

Table 2 — The curated five-document RQ2 set

The two settings differ deliberately and should not be conflated. RQ1 and RQ3 retrieve over a single 22,633-article corpus and report on 222 questions; RQ2 retrieves per document over five codes and reports on 725 questions.

3.2.1 Corpus construction and ground-truth drift

The drift that motivates the curation above is not incidental; it is a measurable property of working with a benchmark whose annotations are a fixed-time snapshot. A parallel corpus-construction effort quantified it directly. Using a custom extraction pipeline — document download, font-size and regular-expression analysis to recover the structural hierarchy, article-level segmentation, and a three-stage normalised match of each article number back to the benchmark — it built a broad corpus of statutory articles from 49 consolidated Justel PDFs, then matched every benchmark article to its current-law counterpart.

The result quantifies the gap between the benchmark and current law. Of the 22,633 benchmark articles, classified by whether their text still matches the benchmark, 58 % are found unchanged, 28 % are present but textually drifted (amended since the benchmark's 2021 snapshot), and 13 % are missing or abrogated, so 86 % are locatable in some form in the current Justel PDFs. Of the located articles, roughly two-thirds are unchanged since the snapshot and one-third have drifted. At the level of whole questions, only 44 % have their ground-truth articles fully present and unchanged in current law, a further 35 % are partially present, and the rest are mixed or absent.

These figures justify two choices in RQ2: curating a clean five-document set on which ground truth is locatable, and reporting a drift-aware recall view alongside the headline metrics (§4.2.5), one that restricts the denominator to ground-truth articles actually present in each document. They also bound the realism of any retrieval evaluation that applies a snapshot benchmark to live regulatory text.

3.3 Query stratification and the 48-question subset

Two stratification layers, computed once and reused across every tier and research question, underpin the failure-mode analyses of Chapter 5.

Six query strata partition the test questions along three binary axes. The first is *single-versus multi-article* questions (one relevant article versus more than one), which surfaces the multi-article recall ceiling. The second is *lexically-aligned versus semantically-paraphrased* questions, taken as the top versus bottom quartile of the question's lexical-match score against its ground-truth article (the middle quartiles are excluded), which surfaces the sparse-versus-dense complementarity. The third is whether a question's ground-truth articles *contain cross-references*, which surfaces cross-reference dependence. The lexical-alignment axis is a *proxy*: it is defined by a BM25-style overlap score rather than a hand-annotated label, and it deliberately contrasts the extremes of the distribution (a limitation noted in §6.5).

The 48-question stratified subset is a deterministic sample drawn to cover every cell of the cross-product of these difficulty axes. It comprises 48 of the 222 test questions (21.6 %) and is held constant across every tier and every research question, so that the autonomous evaluators of RQ3 — too costly to run on the full test set — and the supervised metrics can be compared on an identical set of question–system pairs. The subset stays faithful to the full split: the rank correlation between subset-restricted and full-test supervised metrics exceeds 0.85, so the ordering it induces is a reliable proxy for the ordering on the full set.

3.4 Evaluation metrics

All retrieval is scored with standard binary-relevance information-retrieval metrics, computed by the shared evaluation harness (§3.5). **Recall@k**, the fraction of a question's relevant articles found in the top k , is the primary metric, reported chiefly at $k = 100$ and $k = 10$. Three further metrics capture ranking quality: how highly relevant articles are placed within the top k , not just whether they appear at all. **MRR@k** rewards placing the first relevant article as close to the top of the list as possible. **NDCG@k** rewards placing several relevant articles near the top, discounting each one the further down the list it appears. **MAP@k** rewards a ranking where relevant articles are clustered near the top rather than scattered throughout. **Hit@k** (whether any relevant article appears in the top k) and the count of questions with no relevant article in the top k are reported alongside the headline results. Recall@100 is primary, following the original BSARD paper and because BSARD's high answer multiplicity caps Recall at small cut-offs well below one for many questions;

Recall@10 is the secondary, re-ranking-focused metric, used wherever what matters is the top-of-list surface presented to a downstream reader. Comparisons between systems are tested for significance with paired tests on per-query recall at the relevant cut-off.

3.5 Reproducibility and tooling

Every experiment in the thesis writes a single self-describing result record to a fixed schema: its configuration, the full metric panel, the stratified breakdowns, and the significance test against its anchor. Every number in Chapter 5 is therefore traceable to a stored artifact, available in the uploaded code and dataset repositories. Metric computation is delegated to a shared external evaluation harness rather than reimplemented per study, which keeps the metric definitions identical across all three research questions. The implementation draws on a standard open-source retrieval stack: FAISS (Johnson et al., 2021) for dense vector search, rank-bm25 and SQLite FTS5 for sparse retrieval, and the Hugging Face transformers and sentence-transformers libraries for encoding, which power the Tier 2 dense encoders (CamemBERT, multilingual E5, BGE-M3, and Qwen3). LangChain and LangGraph orchestrate the agentic pipelines of Tier 4 and the RQ2 navigation arms. Language-model inference is served locally through Ollama for the open-weight LLaMA backbone and through the OpenAI API for GPT-4o-mini. Of the methods compared, only the decoder-only embedding model and the Tier 4 language-model runs require a GPU.

With the corpora, stratification, metrics, and infrastructure established, Chapter 4 turns to the methods of the three studies.

4 Methodology

This chapter sets out the methodology of the three studies. They share one organising principle: each comparison is designed so that *one axis varies while everything else is held constant*, so that any measured difference is attributable to that axis. Section 4.1 presents the four-tier retrieval comparison of RQ1, §4.2 the structure-aware approaches of RQ2, and §4.3 the autonomous evaluator panel of RQ3. The corpora, stratification, and metrics are those defined in Chapter 3.

4.1 RQ1 — Retrieval method comparison

RQ1 is operationalised as a *ladder* of four retrieval tiers, each adding exactly one mechanism over the previous tier's best configuration: sparse lexical retrieval (Tier 1), dense semantic retrieval (Tier 2), hybrid fusion of the two (Tier 3), and a language model, first as a relevance scorer and then as an agentic controller (Tier 4). Every tier retrieves over the same article-level units of the canonical BSARD corpus (§3.1), and each tier's winner, chosen on the metrics defined in §3.4, seeds the next, so any difference between tiers reflects the added mechanism rather than a change in chunking.

4.1.1 Tier 1 — Sparse retrieval

Tier 1 establishes the lexical baseline. It covers the standard sparse methods — TF-IDF, BM25 with its Okapi, Plus, and L variants, and the SQLite FTS5 ranker² — in a structured ablation over two orthogonal axes: **text normalisation** (none, stemming, or lemmatisation) and **field weighting** (article text alone versus a variant that also up-weights legal-code and heading metadata). BM25's term-frequency-saturation and length-normalisation parameters are tuned on a held-out validation split, carved out of BSARD's training data (§3.1) and never seen by the test-split evaluation reported in Chapter 5, following the validation protocol of the original BSARD paper

² TF-IDF scores a document by how often a query's terms appear in it, discounted by how common each term is across the whole corpus. BM25 refines this with saturation (repeating a term stops helping after a point) and length normalisation; its Okapi, Plus, and L variants differ only in how that normalisation is tuned. SQLite FTS5 is a full-text-search engine with its own BM25-style ranking built in. For example, for the query 'délai de préavis' (notice period), all these methods rank an article containing that exact phrase highly, but none would recognise an article that instead says 'durée du congé' (length of notice) as relevant, since none of them look beyond the literal words.

(Louis & Spanakis, 2022), and the French stop-word list is overridden to keep tokens that carry legal force — negations and conditional connectives such as *ne*, *pas*, *sans*, *sauf*, whose removal would invert a statutory provision. Tier 1 has two goals: to establish the **lexical ceiling** — the best recall exact-match methods can attain on BSARD — and to identify the strongest sparse configuration as the lexical leg of the Tier 3 hybrid.

4.1.2 Tier 2 — Dense retrieval

Tier 2 asks how far *zero-shot* dense bi-encoders close the vocabulary gap that limits Tier 1. The comparison spans eight configurations across encoder families and scales: CamemBERT (Martin et al., 2020), the French masked-language-model baseline the BSARD paper evaluated zero-shot; a French sentence-transformer; multilingual E5 in base and large sizes (Wang et al., 2024); BGE-M3 (Chen et al., 2024a); a multilingual MPNet as a short-context reference; and a recent decoder-only embedding model (Qwen3), evaluated in both a base and an instruction-tuned variant. Each query and article is embedded independently and ranked by cosine similarity, exact inner-product search over L2-normalised vectors with FAISS (Johnson et al., 2021), with approximate indices avoided so the reported recall reflects the encoder, not index error. Each model's required conventions (query/passage prefixes, pooling strategy) are enforced exactly, since the wrong convention silently degrades the embeddings. The best encoder's metadata-weighted variant is selected on a small stratified training subsample, then evaluated once on the test split. No encoder is fine-tuned on BSARD (§1.5), so the comparison isolates retrieval *architecture* from training and reproduces the BSARD paper's zero-shot CamemBERT result as a reference point.

4.1.3 Tier 3 — Hybrid fusion

Tier 3 fuses the ranked lists of the Tier 1 and Tier 2 winners, whose strengths are complementary — sparse excels on lexically-aligned queries, dense on paraphrased ones. Both legs are frozen at their winners, so any gain is attributable to the fusion mechanism, not a stronger leg. Three fusion families are compared, each with its own hyperparameter grid: **Reciprocal Rank Fusion** (Cormack et al., 2009), combining systems by the reciprocals of their ranks under a single damping constant; **linear score interpolation**, a weighted sum of the dense cosine score and the min-max-normalised sparse score; and **sparse-gated dense re-ranking**, in which the sparse retriever supplies a large pool the dense encoder re-scores.³ Rank fusion is expected

³ For example, if an article ranks 3rd under the sparse system and 15th under the dense system, RRF combines these two positions using the reciprocals of their ranks (1/3 and 1/15, plus a small damping constant), so an article that ranks reasonably well under either

to win, operating on ranks rather than raw scores and so sidestepping score calibration (§2.2). A hybrid counts as a genuine improvement only if it significantly beats the best dense-only system, the significance anchor. The single best hybrid is carried forward as the first stage for every Tier 4 sub-tier — essential to the matched-pool comparison below.

4.1.4 Tier 4 — LLM-Judge and agentic retrieval

Tier 4 asks a two-part question: does a language model improve retrieval over the best non-LLM pipeline (Tier 3), and if so, does the gain come from the *presence* of the model or from the *agentic machinery* built around it? To separate the two, Tier 4 is decomposed into three sub-tiers sharing the same language model and first stage, so the only axis that varies is the mechanism: a non-agentic single-pass relevance scorer (T4.0), an agentic single-evaluator correction loop (T4.1, CRAG), and an agentic multi-step reasoning loop (T4.2, ReAct). The shared backbone is one 8-billion-parameter LLaMA 3.1 model ([Grattafiori et al., 2024](#)) at temperature zero; the shared first stage is the Tier 3 hybrid winner.

T4.0 is not merely a baseline but a necessary control: with it, the differences T4.1 – T4.0 and T4.2 – T4.0 are clean measures of *agentic* value; without it they would be confounded with the value of having any LLM in the pipeline at all. A second principle, the **matched-pool rule**, follows from the fact that a re-ranker re-orders only the candidates it is given, so its recall at depth k is meaningful only when k does not exceed its pool size: comparing an agentic method that sees the top 20 against a scorer applied to the top 50 would confound mechanism with pool depth. Every agentic-versus-non-agentic comparison is therefore made at a matched pool — the agentic loops and a dedicated run of the scorer all seeing the same 20 candidates. This comparison is the source of the headline RQ1 finding (Chapter 5).

T4.0 — the language model as relevance judge. The first stage retrieves a candidate pool, the language model scores each (query, article) pair, and the candidates are re-ranked by that score — methodologically the cross-encoder re-ranker of §2.2 with a prompted LLM in place of the trained model. Two scoring paradigms are compared, a binary relevant/not-relevant judgement and a 0–10 numeric score, with the binary variant taken as canonical for reasons the results make clear.

system scores well overall, without ever needing to compare the two systems' raw, differently-scaled scores directly. Linear interpolation instead compares the scores themselves, after rescaling both to a common 0-1 range.

T4.1 — CRAG. CRAG ([Yan et al., 2024](#)) wraps the first stage in a correction loop: a single collective evaluator call judges whether the retrieved set is legally sufficient and, if not, the query is rewritten ([Ma et al., 2023](#)) and the corpus re-retrieved. Because BSARD is a closed corpus, CRAG's original web-search fallback is inapplicable; this thesis replaces it with intra-corpus query rewriting, relaxing the sufficiency criterion progressively across iterations and rewriting in three stages from a literal statutory rephrasing to aspect-focused reformulations. The evaluation pool is fixed at the matched-pool size.

T4.2 — ReAct. ReAct ([Yao et al., 2023](#)) interleaves reasoning with retrieval actions in a Thought-Action-Observation loop, terminating when the agent judges its evidence sufficient; the final ranked list is recovered deterministically from the accumulated observations rather than the model's free-text recall. An initial version (v1) underperformed its own first stage, spending its step budget on redundant retrievals — an instructive failure in itself. A revised v2 narrowed the observation window to force new query angles, added an overlap guard against redundant retrievals, switched to a function-calling action format to cut parse failures, and added a post-hoc re-rank of the accumulated pool. ReAct v2 is canonical; v1 is retained only as a design-failure reference.

4.2 RQ2 — Structure-aware retrieval

RQ2 asks whether exploiting the hierarchical structure of regulatory documents improves retrieval, operationalised as a comparison of approaches that make progressively heavier use of that structure. All are evaluated *per document* on the PDF-extracted corpus (§3.2) and, crucially, through one common scoring path (§4.2.5). Three primary approaches form a ladder of structural commitment — structure-agnostic, structure folded into the index, and structure as the retrieval mechanism — and a follow-up asks whether the weakest of them can be rescued.

To illustrate: consider a question about the notice period an employer must respect, whose answer lies in Article 37, buried under Book III › Title II › Chapter IV of a labour code. Arm 1 (naive hybrid) simply searches fixed 512-token windows of the whole document for lexical and semantic matches, with no awareness that Article 37 sits under “Chapter IV”. Arm 2A (metadata-enriched) indexes Article 37 together with a short LLM-generated summary and its hierarchy path (“Book III, Title II, Chapter IV: Termination of the employment contract”), so a query that shares little vocabulary with the article itself can still match on the summary or the chapter heading. Arm 2B (PageIndex navigation) instead has a language model read the document's table of contents and choose, first, which title is relevant, then which chapter, then which

article, arriving at Article 37 by narrowing down the hierarchy rather than searching an index. Arm 2C rebuilds this navigation with the full heading stack at each step, so the model chooses among a handful of chapters within Title II rather than among all the code’s chapters at once, and lets an agentic loop revisit branches it initially skipped; CNR then re-ranks the small set of articles this navigation reaches using Arm 2A’s dense encoder, rather than trusting the navigator’s own ordering.

4.2.1 Arm 1 — Naive hybrid baseline

Arm 1 is the control. The RQ1 hybrid backbone is re-pointed at the PDF corpus with no use of document structure: each document is split into fixed sliding-window chunks (512-token window, 256-token stride), retrieved by the same dense-plus-BM25 rank fusion as in RQ1, and the chunk ranking is resolved to articles by crediting each article its best-scoring chunk. It draws on a deeper candidate pool than the structure-aware arms and makes no query-time language-model calls — the structure-blind reference point the others must beat.

4.2.2 Arm 2A — Metadata-enriched retrieval

Arm 2A puts structure in the representation. It keeps the same fusion retriever but changes the indexed unit from an arbitrary window to a *structural node*, an article or finer sub-node recovered from the document hierarchy, and enriches each unit before indexing with LLM-generated metadata (a short summary and salient legal terms) alongside its hierarchy path. Enrichment variants are compared against an unenriched control at two granularities (node and whole-article), and an optional query-time boost/filter stage re-weights candidates by metadata matches. All enrichment is computed once at index time, so Arm 2A, like Arm 1, makes no query-time language-model calls. It answers two questions: whether index-time enrichment improves over the raw node, and whether the query-time boost/filter adds anything on top.

4.2.3 Arm 2B — PageIndex navigation

Arm 2B makes structure the retrieval mechanism. Abandoning vectors and chunking entirely, it compiles each document into a navigable tree and has a language model descend it top-down, choosing which chapters to enter and then which articles, the way a jurist narrows a search. The navigator is the same 8-billion-parameter LLaMA 3.1 model used in RQ1, run locally at temperature zero, and the resulting article list is padded to a fixed depth for scoring. This is the strongest form of the structure hypothesis, that explicit hierarchy navigation can *substitute* for vector

retrieval. The version evaluated here groups each article under its single nearest heading, leaving a large code as a near-flat list of roughly a hundred sibling chapters.

4.2.4 Arm 2C — Agentic deep-tree navigation (follow-up)

Arm 2C asks whether Arm 2B's poor result condemns hierarchy navigation as an idea, or only its flattened tree. It keeps the vectorless premise for the navigation step itself, the same 8B model reading a tree with no embeddings involved in selecting candidates, but repairs two weaknesses. Its CNR extension, introduced below, later re-ranks the small pool this navigation reaches using an embedding model, a distinct step from vectorless navigation rather than a return to it. First, the tree is rebuilt from each article's full heading stack (Book → Title → Chapter → Section → Article), so each decision chooses among a handful of children rather than ninety-odd siblings. Second, the fixed navigation pipeline is replaced by an agentic ReAct loop (§4.1.4) that descends to a variable depth, followed by a re-rank of the navigated pool. This shipped configuration is termed simple navigation.

A further synthesis, **CNR (Corrective Navigation and Re-ranking)**, then attacks the two residual failure modes that simple navigation leaves: it *reaches* too little of the relevant material, and it *orders* poorly what it does reach. CNR adds two independent, additive levers. A corrective navigation loop, in the spirit of CRAG (§4.1.4), re-seeds the search frontier from the branches the greedy descent pruned, fixing reach. An embedding re-rank then replaces the language model's ordering with Arm 2A's encoder applied to the navigated pool only (no full vector index), fixing ranking. Arm 2C thus doubles as a constructive test of whether navigation's deficit is one of coverage or of ranking.

4.2.5 The common scoring path

The arms were originally evaluated by different means, against subtly different ground-truth definitions and at different pool depths, so their native numbers are not directly comparable. Here every approach is re-scored through one identical metric path against a single per-document ground truth: the same relevance definition, the same resolution of retrieved units to articles, and the same metric formulas, so that any difference reflects the approach rather than the scoring. No retrieval was re-run; only the scoring was unified. One asymmetry is kept, and footnoted where it bites: the candidate-pool depths still differ, since the naive baseline uses a deeper pool than the structure-aware arms, so recall at the deepest cut-off structurally favours the baseline. Because the extracted-PDF setting carries ground-truth drift (§3.2), a *drift-aware* recall view, restricting the denominator to ground-truth articles actually

present in each document, is reported alongside the headline numbers as a sensitivity check.

4.3 RQ3 — Autonomous evaluator panel

RQ3 asks whether retrieval quality can be assessed without ground-truth relevance judgements, through a controlled validation: ground truth exists in BSARD but is deliberately withheld while a panel of reference-free evaluators scores each retrieval system, the systems are ranked by each evaluator, and those rankings are compared against the supervised one. Agreement would establish the panel as a trustworthy substitute where annotations are unavailable. The RQ1 systems are the test bed, spanning a wide quality range on the same questions and ground truth. The panel is applied to the 19 first-stage retrievers (sparse and dense) on the fixed 48-question stratified subset (§3.3); the downstream hybrid and agentic systems are excluded, since they re-rank the first-stage pools rather than introduce new retrieval signal, so the question is studied most cleanly at the first-stage layer.

4.3.1 The evaluators

The panel comprises four reference-free evaluators (§2.6), all using the same judge language model (gpt-4o-mini) and all consuming the top-10 retrieved articles per query. gpt-4o-mini is each evaluator's own default judge model, and it is kept consistent across the panel so that every evaluator is read as its own benchmark, rather than mixing judge models between them. **UMBRELA** ([Upadhyay et al., 2024](#)) assigns a graded 0–3 relevance label to each (query, article) pair, in the TREC relevance-assessment tradition. **eRAG** ([Salemi & Zamani, 2024](#)) tests each retrieved article's utility by whether it enables a grounded answer. **RAGAS-WA** adapts a context-precision metric ([Es et al., 2024](#)) to the retrieval-only setting by supplying a hypothetical answer generated from the query — in the spirit of HyDE ([Gao et al., 2023a](#)) — in place of the missing reference answer. **RAGAS-WB** is the same metric using the query itself as the proxy, included only as a diagnostic to isolate the contribution of the HyDE step. The first three are treated as scored components; RAGAS-WB is diagnostic only.

4.3.2 The gold-ceiling diagnostic

A system-level rank correlation tells whether the panel orders systems like the supervised metrics, but not whether its *absolute* scores are meaningful. The gold-ceiling diagnostic addresses the second question. Each evaluator is applied to a *gold-*

only retrieval — one whose top-10 contains only ground-truth-relevant articles — to establish the score it assigns to perfect retrieval. An evaluator that rates any real retriever *above* this oracle does not recognise the ground truth as the correct answer, so its absolute scores cannot be trusted as a measure of correctness, even if its rankings remain useful. This diagnostic is what later separates the trustworthy component from the others.

4.3.3 Validation methodology

The panel is validated by comparing *each evaluator* against the supervised metrics — fundamentally a comparison *among* the evaluators, not an endorsement of any one. The primary quantity is the system-level rank correlation across the 19 systems, Spearman ρ with bootstrap confidence intervals, between each evaluator's ordering and the supervised one, asking which evaluator's ranking best matches ground truth, and on which supervised axis (Recall@10, NDCG@10, and the others). Two analyses complement it: a within-system, per-query agreement analysis testing whether an evaluator that ranks systems well also tracks relevance at the individual-query level, and the gold-ceiling diagnostic of §4.3.2 testing whether each evaluator's absolute scores are trustworthy. Because one evaluator alone emits graded relevance labels of the same type as the human qrels, those labels are also fed back through the supervised metric pipeline as a secondary, document-level cross-check. Together these determine which evaluator — if any — can stand in for supervised evaluation, and where its scores can be trusted.

Chapter 5 reports what each of these controlled comparisons yields.

5 Results

This chapter reports the empirical outcomes of the three studies whose methods were set out in Chapter 4. It is confined to results; their interpretation and the implications for system design are reserved for Chapter 6. Section 5.1 reports the RQ1 retrieval ladder and its matched-pool agentic comparison, §5.2 the RQ2 structure-aware comparison, and §5.3 the RQ3 evaluator comparison. All RQ1 and RQ3 figures are on the 222-question BSARD test split (§3.1), and all RQ2 figures on the five-document, 725-question PDF set (§3.2). Significance is assessed with paired tests on per-query recall (§3.4).

5.1 RQ1 — Retrieval method comparison

5.1.1 Tier 1 — Sparse retrieval

The strongest sparse configuration, a lemmatised, length-normalisation-tuned BM25, reaches $\text{Recall@10} = 0.265$ and $\text{Recall@100} = 0.521$ (stemming, the third normalisation option in the §4.1.2 ablation, was also evaluated but consistently underperformed lemmatisation, so it does not appear among the representative configurations below); a metadata-weighted variant attains the highest sparse Recall@100 at 0.531. No BM25 variant beats the paper-aligned default by a significant margin on Recall@10 (all $p > 0.05$), so 0.265 is best read as the *lexical ceiling* on this benchmark rather than the product of tuning, and the paper's BM25 result is reproduced faithfully at the rank-aware level (MRR@100 within ± 0.0004). The decisive observation is stratified. On lexically-aligned queries the best sparse system reaches $\text{Recall@10} \approx 0.52$, but on semantically-paraphrased queries it collapses to ≈ 0.04 , a fourteen-fold gap that no normalisation, field weighting, or hyperparameter choice closes, and the direct motivation for the dense tier.

System	Normalisation	Fields	R@10	R@50	R@100	MRR@100	NDCG@10
Tuned BM25 (Okapi, $k_1=1.5$, $b=0.25$)	lemma	text	0.265	0.463	0.521	0.263	0.222
BM25 (Okapi) + metadata	lemma	text+meta	0.257	0.473	0.531	0.252	0.210
BM25 Plus	lemma	text	0.246	0.435	0.509	0.255	0.207
BM25 L	lemma	text	0.175	0.315	0.404	0.134	0.117
TF-IDF + metadata	lemma	text+meta	0.200	0.426	0.515	0.200	0.153
FTS5 (default)	none	text	0.237	0.388	0.470	0.237	0.190
Paper-aligned BM25 (anchor)	none	text	0.248	0.419	0.482	0.246	0.201

Table 3 — Sparse retrieval: representative configurations

5.1.2 Tier 2 — Dense retrieval

Zero-shot dense retrieval closes a substantial part of that gap. The best encoder, multilingual E5-large with metadata weighting, reaches Recall@10 = 0.342 and Recall@100 = 0.622, the latter a significant improvement over the sparse anchor ($p < 0.001$); on the paraphrased stratum its Recall@10 more than triples the best sparse figure (≈ 0.14 vs ≈ 0.04). Three secondary findings stand out. Model scale matters: within each family the large encoder clearly beats the base one, most dramatically for CamemBERT, whose large variant reaches Recall@100 = 0.583 while its base variant collapses to ≈ 1.7 %. The strongest systems are large multilingual encoders, though the large French-specific encoder remains competitive, just behind them. The CamemBERT-base collapse faithfully reproduces the zero-shot CamemBERT result of the original BSARD paper; it reflects the difficulty of using a small, untuned masked-language-model encoder off the shelf, rather than a limitation of contextual encoders in general, since the large variant performs well. The best zero-shot encoder reaches ≈ 62 % Recall@100 against the paper's fine-tuned ceiling of ≈ 72 %, closing the majority of the zero-shot-to-fine-tuned gap with multilingual pretraining alone. Dense retrieval does not dominate, though: even the best encoder is only roughly on par with sparse on lexically-aligned queries (0.535 vs approximately 0.52), the complementarity the hybrid tier exploits. Qwen3, a decoder-only encoder added later as an additional baseline and not part of the winner selection that carried forward into the hybrid tier, does clearly exceed sparse on this stratum (0.567 to 0.590 across its variants).

System	Size	Weighting	R@10	R@50	R@100	MRR@100	NDCG@10
Multilingual E5-large	560M	text+meta	0.342	0.563	0.622	0.322	0.270
Qwen3-Embedding-0.6B (instruction-prefixed)	600M	text	0.339	0.512	0.597	0.328	0.268
Multilingual E5-large	560M	text	0.296	0.486	0.594	0.300	0.230
BGE-M3	568M	text	0.314	0.509	0.592	0.303	0.242
CamemBERT-large	335M	text	0.307	0.507	0.583	0.252	0.215
Multilingual E5-base	278M	text	0.225	0.418	0.491	0.245	0.182
Multilingual MPNet	278M	text	0.211	0.321	0.405	0.197	0.154
CamemBERT-base	110M	text	0.008	0.016	0.017	0.022	0.010

Table 4 — Dense retrieval: representative zero-shot configurations

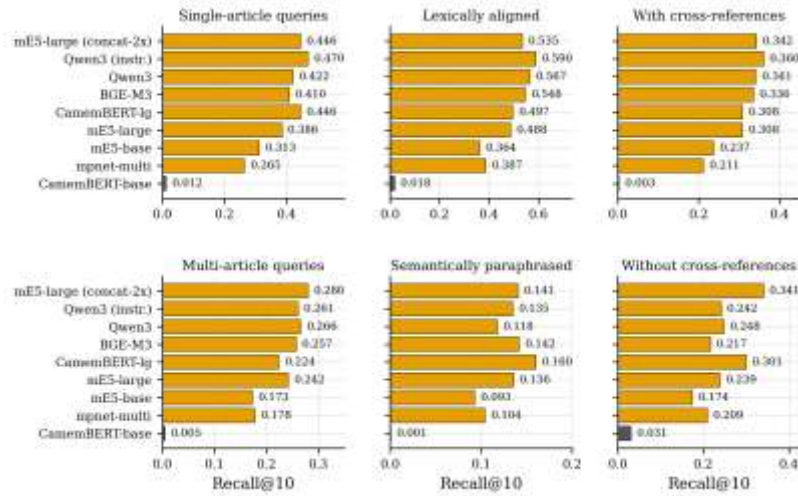


Figure 1 — Dense retrieval: stratified Recall@10 by encoder

5.1.3 Tier 3 — Hybrid fusion

Fusing the two legs confirms the complementarity. Reciprocal Rank Fusion is the strongest mechanism, reaching Recall@10 = 0.402, a significant 6.0-point gain over the best dense system ($p = 0.015$), and Recall@100 = 0.651, the best non-LLM result in the study. It is also the only system to win both strata at once: Recall@10 ≈ 0.72 on lexically-aligned queries, higher than either modality alone, and ≈ 0.13 on paraphrased queries, matching dense. Rank fusion clearly outperforms linear score interpolation and sparse-gated dense re-ranking, consistent with its avoidance of score calibration. This hybrid is carried forward as the first stage for every Tier 4 sub-tier.

Fusion	Setting	System	R@10	R@50	R@100	MRR@100	NDCG@10
RRF	k=60	Hybrid (BM25 + E5-large)	0.402	0.580	0.651	0.350	0.309
Linear	$\alpha=0.9$	Hybrid (BM25 + E5-large)	0.384	0.569	0.622	0.340	0.302
SGDR	K=1000	Hybrid (BM25 + E5-large)	0.365	0.542	0.592	0.337	0.285
—	—	Dense anchor (E5-large + metadata)	0.342	0.563	0.622	0.322	0.270

Table 5 — Hybrid fusion: best configuration per family

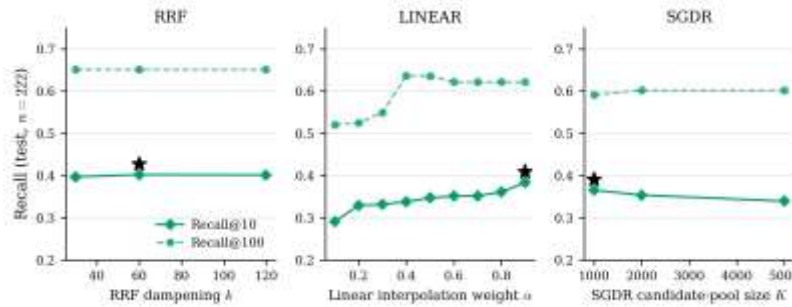


Figure 2 — Tier 3 hybrid hyperparameter sweeps (RRF, linear, SGDR)

5.1.4 Tier 4 — LLM-Judge and agentic retrieval

Tier 4 places the language model over the frozen hybrid first stage, first as a single-pass relevance scorer (T4.0), then as the controller of two agentic loops (T4.1 CRAG, T4.2 ReAct). Table 6 collects the canonical configurations; the three sub-tiers are then examined in turn, each measured against the same hybrid first stage and, for the loops, against the matched-pool scorer.

Sub-tier	Mechanism	Pool	R@10	R@20	R@50	R@100	MRR@100	NDCG@10	Hit@10
—	Hybrid (RRF) first stage	full	0.402	0.465	0.580	0.651	0.350	0.309	0.586
T4.0	LLM-Judge (binary)	top-50	0.445	0.518	0.580	—	0.406	0.358	0.653
T4.0	LLM-Judge (binary)	top-20 (matched)	0.435	0.465	0.465	—	0.396	0.352	0.635
T4.1	CRAG	top-20 + loop	0.426	0.465	0.582	0.654	0.401	0.346	0.626
T4.2	ReAct v2	top-20 + loop	0.426	0.465	0.468	—	0.378	0.335	0.622

Table 6 — Tier 4 canonical results

T4.0 — the language model as relevance scorer. Introducing the binary LLM-Judge over the hybrid first stage is the one Tier 4 step that clearly pays: Recall@10 rises to 0.445 (+0.043 over the hybrid, $p = 0.016$), the highest in the study. Two controls fix the configuration. Binary scoring far outperforms the 0–10 numeric variant (0.445 vs 0.272); a plausible explanation is that a model this size struggles to produce well-calibrated continuous scores, though the deeper reasons for this gap merit further study. In practice, the numeric variant also proved harder to prompt reliably, requiring repeated prompt revisions when evaluating questions over the Code Pénal, and given these unsatisfying results, the 0–10 scoring approach was not carried forward into later experiments. and the same scorer over a sparse first stage reaches only 0.362, first-stage strength dominates, and the scorer cannot recover what the first stage never surfaces. The depth curve makes the mechanism visible (Figure 3, left): the judge lifts the first stage at the top of the list, then plateaus once k reaches its candidate pool. Stratified, the lift is everywhere (Figure 3, right), the binary hybrid

judge beats the reference in five of the six query strata, including the semantically-paraphrased cases; the one exception is the lexically-aligned stratum, where RRF (0.716) still edges out the judge (0.704 at top-50), the one regime where the hybrid it is built on is hardest to beat.

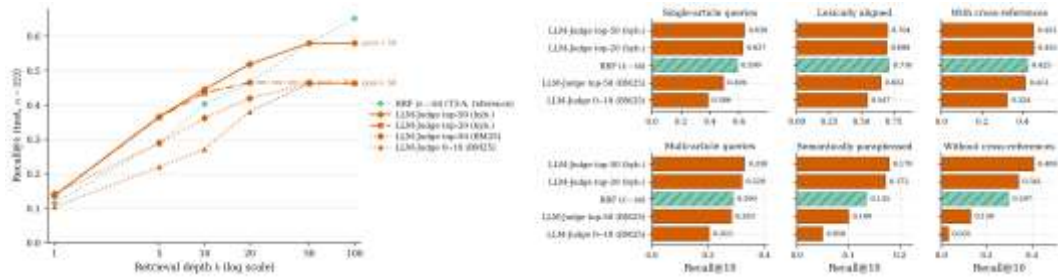


Figure 3 — T4.0 LLM-Judge: Recall@k by depth and stratified Recall@10

T4.1 — CRAG. The corrective loop clears the non-LLM hybrid baseline (Recall@10 = 0.426) but not the scorer. Its contribution is *depth*, not top-rank quality: it is the only Tier 4 system whose recall keeps climbing past the matched pool, reaching Recall@100 = 0.654 as the correction loop expands the candidate set (Figure 4, left). Per stratum it draws level with the matched-pool scorer rather than ahead, it exceeds it in none (Figure 4, right), and, as everywhere in Tier 4, the first stage dominates: the hybrid-fed CRAG far outstrips the BM25-fed one.

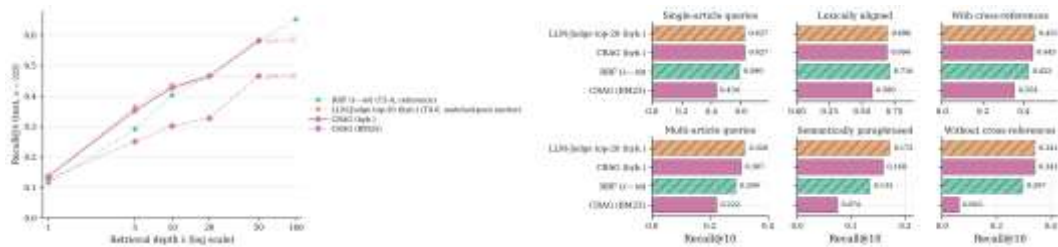


Figure 4 — T4.1 CRAG: Recall@k by depth and stratified Recall@10

T4.2 — ReAct. The reasoning loop is a story of repair. The initial version collapsed to Recall@10 = 0.277, below its own first stage, an instructive failure of free-text multi-step agents on an 8B model, and the v2 redesign (narrowed observation window, overlap guard, function-calling, post-hoc re-rank) recovers it to 0.426 (Figure 5, left). Once recovered, ReAct v2 matches the matched-pool LLM-Judge and the CRAG peer but surpasses neither, in aggregate or in any single stratum (Figure 5, right). The head-to-head agentic-versus-scorer comparison, consolidated across all tiers, is the subject of §5.1.5.

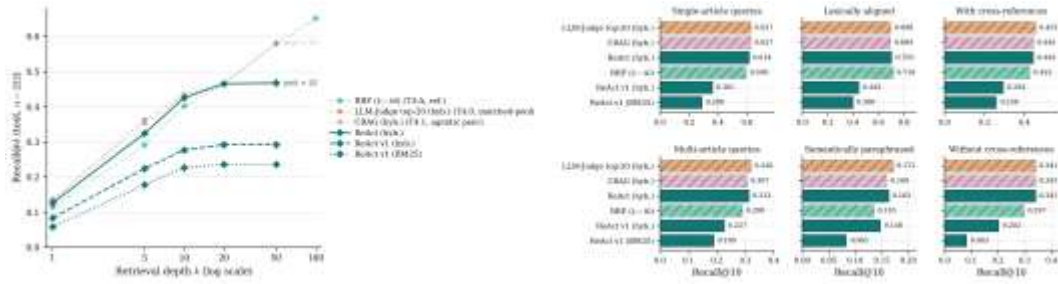


Figure 5 — T4.2 ReAct: Recall@k by depth and stratified Recall@10

5.1.5 Cross-tier synthesis and the matched-pool agentic comparison

Read end to end, the four tiers form a monotonic, statistically significant ladder on Recall@10: 0.265 $\rightarrow$ 0.342 $\rightarrow$ 0.402 $\rightarrow$ 0.445, each step a significant gain over the previous tier's anchor ($p < 0.05$). The largest single step is sparse $\rightarrow$ dense (+0.077), and the peak sits at the non-agentic LLM-Judge. Figure 6 traces the ladder across retrieval depth: the first-stage tiers keep climbing with k , while the re-rankers plateau once k reaches their candidate pool.

Tier	Best system	R@10	R@100
1 — sparse	Tuned BM25	0.265	0.521
2 — dense	Multilingual E5-large + metadata weighting	0.342	0.622
3 — hybrid	Reciprocal Rank Fusion	0.402	0.651
4 — LLM-Judge (non-agentic)	Binary LLM re-ranker (top-50)	0.445	—

Table 7 — The RQ1 retrieval ladder (best system per tier)

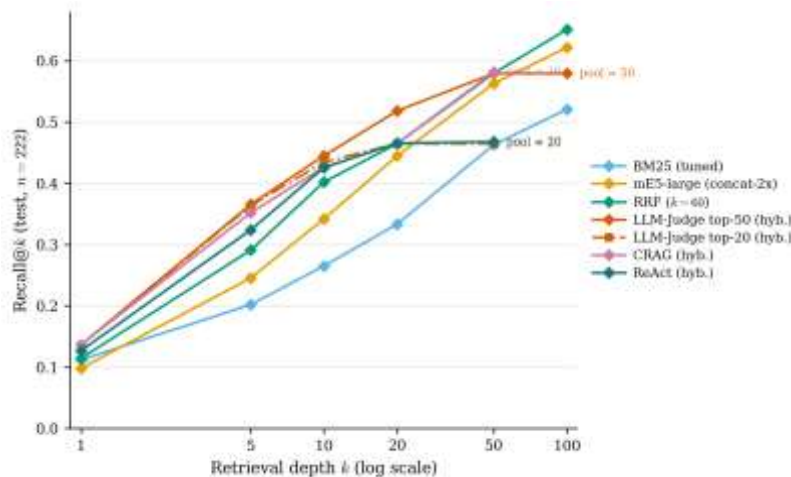


Figure 6 — Recall@k across the four-tier retrieval ladder

The question the tier was built to settle is whether the *agentic mechanism* adds anything over the single-pass scorer, and the figures answer it directly. Both agentic methods land at Recall@10 = 0.426, in line with the LLM-Judge, not beyond it. Across

the six query strata the four LLM systems cluster together, with neither CRAG nor ReAct separating from the scorer in any cell (Figure 7, left); and along both of BSARD's difficulty axes, rising answer multiplicity and falling lexical overlap, the same systems track one another, the agentic loops offering no advantage where the hardest cases lie (Figure 7, right). The exact margins confirm the visual reading (Table 8): against the matched top-20 scorer each agentic method is 0.009 lower, and against the best top-50 scorer 0.019 lower, neither difference significant.

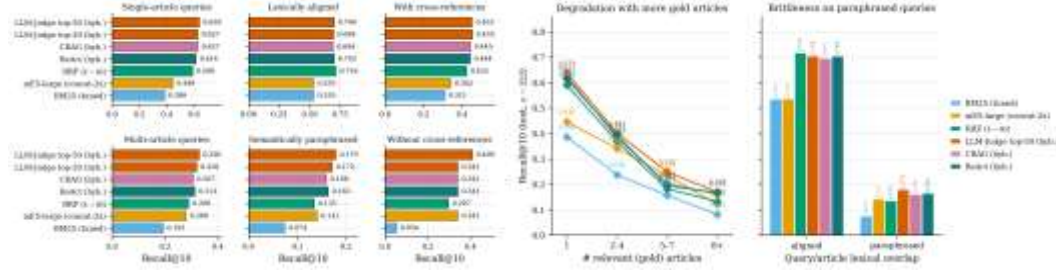


Figure 7 — Stratified Recall@10 and the two difficulty axes, canonical systems

Agentic method (R@10 = 0.426)	vs matched top-20 (0.435)	vs best top-50 (0.445)
CRAG	$\Delta -0.009$ (ns)	$\Delta -0.019, p = 0.163$ (ns)
ReAct v2	$\Delta -0.009$ (ns)	$\Delta -0.019, p = 0.222$ (ns)

Table 8 — Agentic versus non-agentic comparison

Read as an ablation, the two steps separate cleanly: the non-LLM hybrid $\rightarrow$ LLM-Judge step is a real, significant +0.043, while the LLM-Judge $\rightarrow$ agentic step is a non-significant -0.009 to -0.019 . The retrieval benefit at the top of the ladder is attributable to the *presence of the language model as a scorer*, not to the agentic machinery built around it.

The gains localise predictably across strata. The lexical-alignment gap closes at the hybrid tier (lexically-aligned Recall@10 rising to ≈ 0.72); the paraphrase gap closes at the dense tier and again at the LLM-Judge, but not at the agentic methods; and multi-article questions improve monotonically across tiers without ever clearing Recall@10 ≈ 0.33 , the structural recall ceiling imposed by BSARD's high answer multiplicity (§3.1).

The agentic methods also cost more, in language-model work rather than wall-clock time: CRAG runs an extra correction iteration on roughly one query in twenty-five (1.04 iterations per query on average), and ReAct expends 7.85 reasoning steps per query, while the non-agentic scorer makes a single fixed pass and the non-LLM tiers make none. Combined with the null matched-pool result, this makes the single-pass LLM-Judge the most cost-effective operating point of the pipeline. Wall-clock latency is deliberately not reported as a comparable axis: the systems ran on heterogeneous hardware, which would make any latency comparison misleading.

5.2 RQ2 — Structure-aware retrieval

RQ2's results are reported on the five-document, 725-question PDF set (§3.2), every approach re-scored through the common path of §4.2.5. The headline is straightforward: lightweight index-time metadata enrichment is the best approach overall, the structure-agnostic baseline is a strong and robust comparator, and vectorless navigation is dominated, though a follow-up (§5.2.4) shows its deficit is one of ranking rather than coverage, recoverable in part at heavy cost. The four arms are examined in turn, the cross-arm headline first, then each arm's mechanism.

5.2.1 Cross-arm headline — structure helps ranking, not coverage

Table 9 reports the headline metrics, micro-averaged over the 725 questions. The central result is that structure helps *ranking*, not *coverage*. On top-10 coverage the metadata-enriched approach and the naive baseline are effectively tied: under binary relevance the metadata approach leads only narrowly (R@10 0.482 vs 0.472), a statistically insignificant gap (136 vs 130 question-wins) that even reverses under the drift-aware *effective* ground truth, recall scored only over the gold articles actually present in each extracted document (§4.2.5), where the naive baseline edges ahead (0.483 vs 0.472). The metadata approach's clear advantage is in *ranking*: it places the first relevant article at a median rank of 2, lifting MRR@10 to 0.460 (from 0.348) and nDCG@10 to 0.398 (from 0.348). The naive baseline leads on Recall@100 (0.858 vs 0.807), partly a structural effect of its deeper candidate pool (200 vs 100). Vectorless PageIndex navigation trails every vector arm by a wide margin on all metrics (R@10 0.185), and the two agentic deep-tree configurations sit between it and the vector arms; §5.2.4 returns to them.

Arm	R@10	Hit@10	R@100	MRR@10	nDCG@10
Arm 1 — Naive (RRF over chunks)	0.472	0.629	0.858	0.348	0.348
Arm 2A — summary-node (canonical)	0.482	0.691	0.807	0.460	0.398
Arm 2B — PageIndex (LLM nav)	0.185†	0.265	0.404	0.184	0.156
Arm 2C — simple nav	0.263	0.383	0.463	0.241	0.220
Arm 2C — CNR	0.379	0.556	0.589	0.371	0.321

Table 9 — Cross-arm headline metrics (725 questions)

†PageIndex R@100 is mostly deterministic padding (§5.2.3).

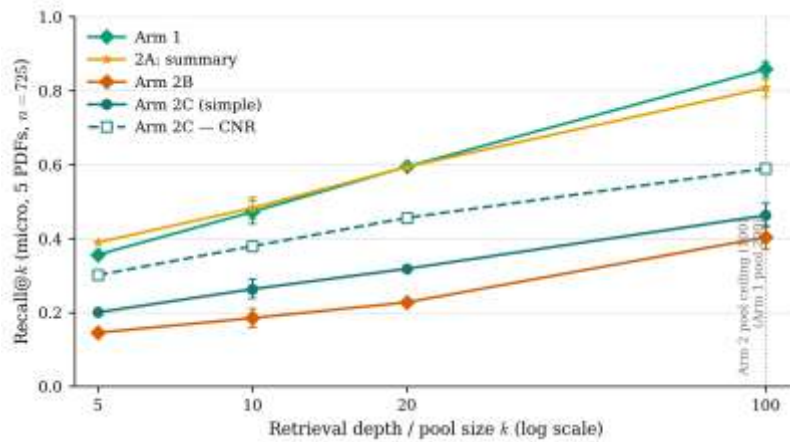


Figure 8 — Recall@k across the structure-aware arms

5.2.2 Arm 2A — where the lift comes from, and what drives difficulty

Two analyses locate the metadata advantage and bound it. First, the lift comes from *index-time enrichment, not query-time re-weighting* (Figure 9, Table 10). The summary-enriched node beats the unenriched raw node by +0.027 Recall@10, and it is the only enrichment that moves the metric: the French metadata headers and the optional query-time boost/filter stage are byte-identical to the raw configuration. Tellingly, the summary is the one signal added in a different language (English) from the French corpus. Second, granularity is a trade-off: the finer node unit buys ranking and recall but fragments the retrieved text (about 29% of retrieved gold articles come back as less than half their text), while the whole-article unit gives complete coverage at the cost of roughly three Recall@10 points.

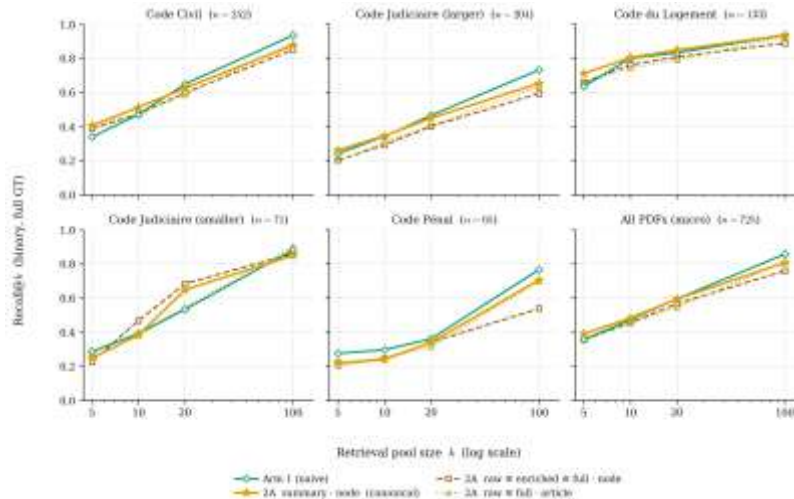


Figure 9 — Arm 2A: the six metadata variants

Variant (unit)	R@10	Hit@10	R@100	P@10	Coverage	% fragmented
summary · node (canonical)	0.482	0.691	0.807	0.147	0.724	28.6
raw · node	0.455	0.692	0.759	0.129	0.713	29.4
raw · article	0.450	0.674	0.810	0.114	1.000	0.0

Table 10 — Arm 2A within-arm: enrichment and granularity

A further analysis shows the dominant difficulty axis is the *question*, not the *approach* (Figure 10, right). Splitting by answer cardinality, the two competitive arms move together, on single-gold questions the metadata approach reaches Recall@10 ≈ 0.57 and the naive baseline ≈ 0.55 , while on multi-gold both fall to ≈ 0.41 – 0.42 , every arm except Arm 2C dropping ≈ 0.13 – 0.14 from single- to multi-article questions ($n = 315$ vs 410). The metadata approach's slim edge is concentrated on single-gold questions, exactly where cleaner ranking pays off; on multi-gold it is indistinguishable from the naive baseline. PageIndex, by contrast, is dominated in both buckets ($\approx 0.24 \rightarrow 0.14$), so its deficit is not a multi-article artefact. The per-document view (Figure 10, left) reinforces the reading: both vector arms recover most gold by $k = 100$, so their top-10 shortfall is one of ranking depth, not coverage.

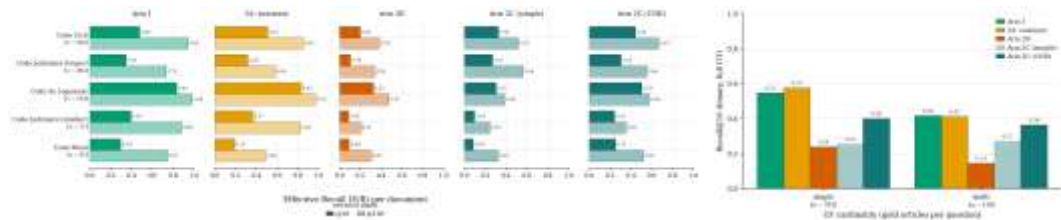


Figure 10 — Per-document recall and Recall@10 by answer cardinality, by arm

5.2.3 Arm 2B — why PageIndex collapses

PageIndex's collapse is worth dissecting, because its headline recall flatters it. Arm 2B reaches only Recall@10 = 0.185, under half the vector arms, and is worse on every metric, with significantly 5.8 language-model calls per query, and resulting in the end with less than half the recall. Its recall is moreover *padding-propped*, not navigated (Figure 11, left): the navigator's own exposed picks recover only 0.137, and the nominal Recall@100 of 0.404 is mostly the deterministic chapter- and law-level padding the pipeline appends, a random fill recovers about 74% of that padding gain. Most of what looks like coverage is an artefact of padding, not localisation.

The miss mechanism is *wrong-chapter selection* (Figure 11, right). Of all 2,874 gold articles, 79.9% sit in chapters the 8B navigator never selected; only 11.2% are right-chapter but wrong-article, and just two articles (0.1%) are absent from the tree, the same two that Arm 2A's index misses, a shared corpus gap rather than a navigation failure. The navigator commits confidently to the wrong subtree: 98% of queries take

a "sufficient" exit at the first iteration, the exposed head is shallow and noisy (median 4 picks, precision 0.091), and cross-reference following almost never fires. So neither the corpus nor the index is the problem; the bottleneck is the one-shot chapter choice over a flattened, roughly hundred-way tree. That diagnosis motivates the deep-tree follow-up of §5.2.4.

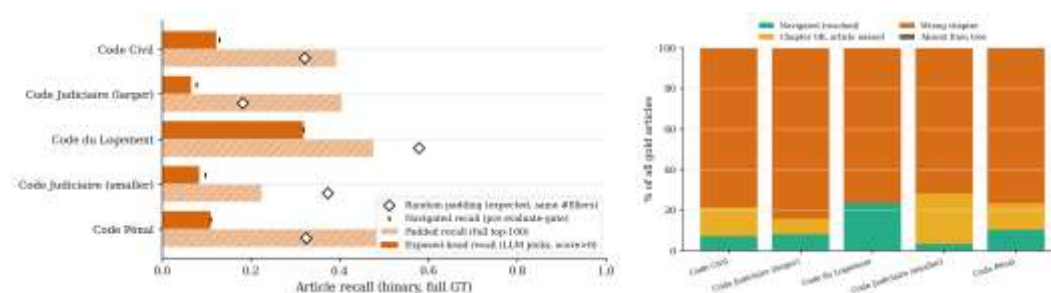


Figure 11 — Arm 2B: navigated-vs-padded recall and the wrong-chapter miss mechanism

5.2.4 Arm 2C — rescuing navigation with a deeper tree

Arm 2C asks whether PageIndex's collapse condemns hierarchy navigation as an idea or only its flattened tree. Rebuilding the full hierarchy and navigating it with an agent (simple navigation) beats PageIndex corpus-wide, Recall@10 0.263 vs 0.185, +42%, and the gain concentrates exactly where the flattened tree hurt most: the Code Civil (+61%) and the larger Code Judiciaire (+122%), the two largest, most deeply structured codes (Figure 12). Much of 2B's collapse was therefore the tree, not the idea. But simple navigation still falls far short of the vector arms (0.263 vs ≈ 0.48) and beats PageIndex on only two of the five documents.

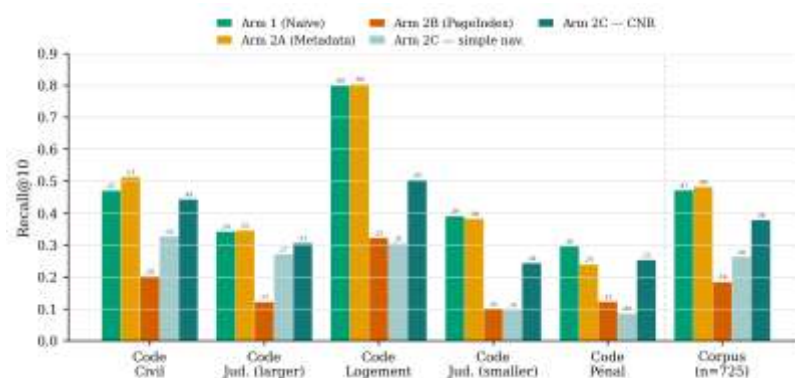


Figure 12 — Per-document Recall@10 across all five arms

Stripping the deterministic padding both navigators lean on splits the deficit cleanly in two (Figure 13, left). On *reach*, the rebuilt deep tree dominates PageIndex on every document (Code Civil 0.47 vs 0.13), surfacing three-to-six times the gold; but on

selection, ordering the reached pool into the top ten, the vectorless 8B is worse, empty-selecting most queries on the harder codes. The residual miss is as much coverage as ranking: 48% of gold is never reached at all, pruned in a branch the greedy descent never revisits (Figure 13, right). The index was never the limit, only two of 2,874 gold articles are genuinely absent.

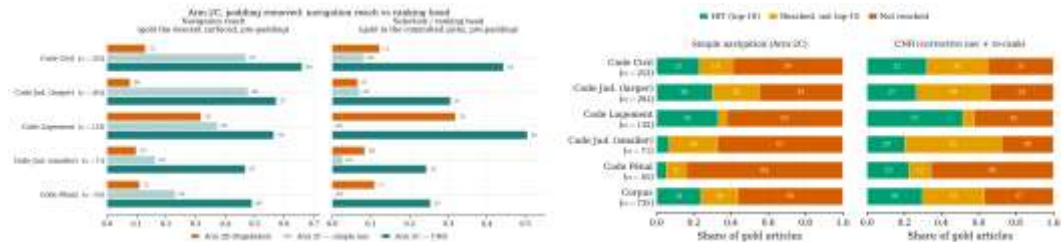


Figure 13 — Arm 2C: reach-vs-selection (padding removed) and gold-article fate

CNR attacks the two failures with two independent, additive levers. A corrective navigation loop, in the spirit of CRAG, re-seeds the search frontier from the branches the greedy descent pruned, lifting padding-free reach substantially ($0.49 \rightarrow 0.80$ on the hard queries). An embedding re-rank then replaces the 8B's ordering with Arm 2A's encoder over the navigated pool only, no full index. On the Code Civil the two levers form a clean ladder (Figure 14): the shipped 8B re-ranker scores 0.332, corrective navigation alone is inert under the 8B ranker (0.335), the embedding re-ranker alone reaches 0.391, and the two together reach 0.443. Neither lever suffices alone, the corrective reach converts to rank only once the embedding ranker is in place. A component view of simple navigation tells the complementary story (Table 11): the deep tree alone barely clears PageIndex (0.166), native metadata fixes selection (0.214), and the re-rank reorders the reached pool into the top ten (0.327).

Both levers compound: navigation reach + embedding ranking

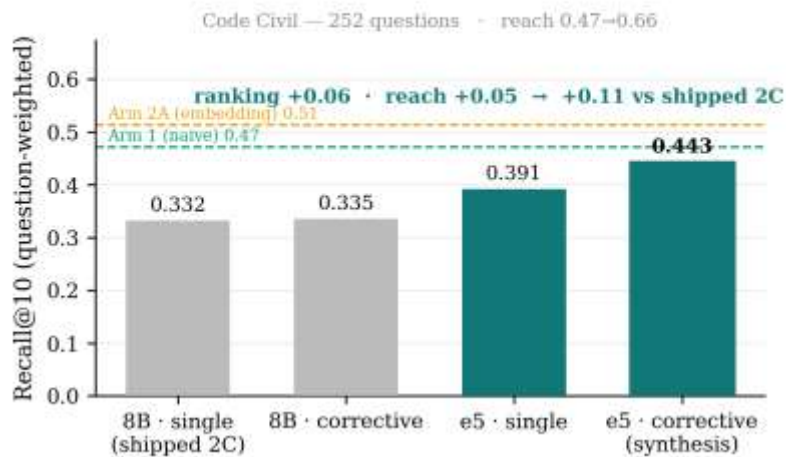
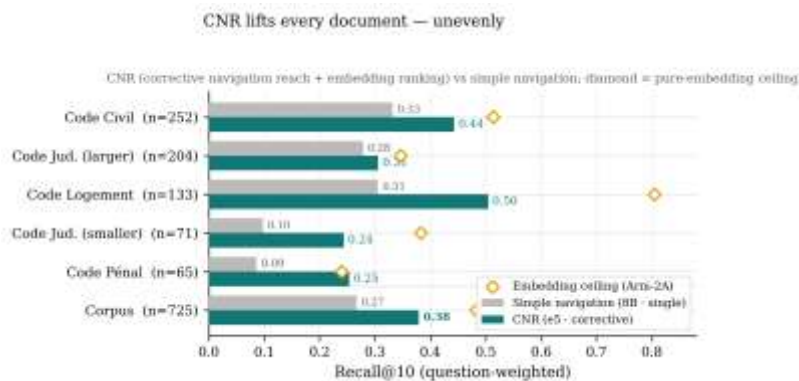


Figure 14 — Code Civil: the CNR lever ladder

Configuration	R@10	R@100	Hit@10	Empty-sel. q
Deep tree (labels only)	0.166	0.450	0.230	158 / 252
+ metadata (enriched)	0.214	0.515	0.286	127 / 252
+ re-rank (= simple nav)	0.327	0.519	0.409	121 / 252

Table 11 — Arm 2C component ablation (Code Civil)

The synthesis generalises: CNR lifts Recall@10 on every document, from 0.267 to 0.379 corpus-wide⁴ (+42%, decomposing into +0.072 from reach and +0.040 from ranking), and with the full metric battery recovered reaches MRR@10 0.371 and nDCG@10 0.321 (Figure 15). Most notably it becomes competitive with vector retrieval on the structurally hardest, least lexical code, beating the metadata-enriched approach on the Code Pénal (0.254 vs 0.240), a deep offence taxonomy where the query never names its chapter, while remaining far behind on the lexically trivial Code du Logement (0.505 vs 0.805), where embedding retrieval is near-perfect and navigation is the wrong tool. The verdict is two-sided (Figure 16). Because only 2 of 2,874 gold articles are genuinely absent, the bottleneck was never indexing but ranking, which CNR demonstrates constructively by converting reach into rank. Yet even with CNR, vectorless navigation trails both vector arms corpus-wide (0.379 vs $\approx$ 0.48), wins only where lexical and dense matching are weakest, and pays the heaviest query-time cost of any approach, about 25 language-model calls per query against the vector arms' zero. The lesson: add structure to rank and reason better, not to widen coverage; navigation earns its keep on hard, low-lexical codes, not where keywords already win.



⁴ This 0.267 baseline is a fresh re-run of simple navigation, measured alongside the CNR conditions in the same batch for a fair comparison, rather than reused from the earlier 0.263 figure; the two differ slightly because the navigator's LLM-driven decisions are not perfectly reproducible run to run.

Figure 15 — Per-document Recall@10: simple navigation versus CNR, with the metadata ceiling

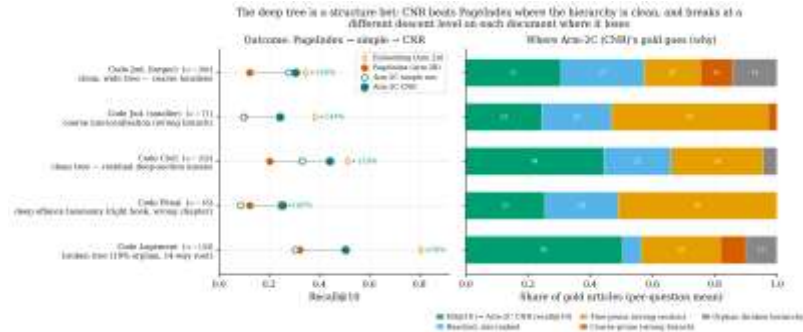


Figure 16 — Why Arm 2C wins on some documents and not others

5.3 RQ3 — Autonomous evaluator comparison

RQ3 compares the three reference-free evaluators against the supervised metrics, over the 19 first-stage retrievers on the 48-question subset (§3.3). The question throughout is which evaluator, if any, can stand in for supervised evaluation, and where its scores can be trusted. Three analyses answer it in turn: whether the evaluators *rank* systems like the supervised metrics (§5.3.1), where that agreement breaks down (§5.3.2), and whether their absolute *scores* can be trusted (§5.3.3). RAGAS-WB, the query-only diagnostic variant introduced in §4.3.1, is omitted here because its scores collapsed toward zero and carried no meaningful signal, confirming that the HyDE-based RAGAS-WA is the version worth reporting.

5.3.1 System-level rank agreement

Across the 19 systems, all three evaluators correlate positively with the supervised ranking, but eRAG is clearly the strongest (Table 12, Figure 17): its system-level Spearman ρ with supervised Recall@10 is 0.79 and with NDCG@10 0.85, against 0.74 / 0.59 for UMBRELA and 0.74 / 0.62 for RAGAS-WA. Ranking is not the same as judging, though. At the level of individual (query, article) pairs the evaluators are weak classifiers against the human relevance labels, Cohen's $\kappa \leq 0.33$ (eRAG 0.33, UMBRELA 0.23), even where raw agreement looks high (eRAG's document accuracy of 0.92 reflects the heavy non-relevant majority, not discrimination), and the within-system, per-query rank agreement is substantially lower than the system-level figure (median $\rho \approx 0.41$ –0.42 for UMBRELA and eRAG, ≈ 0.23 for RAGAS-WA). The evaluators are usable as *system rankers*, not as per-document relevance oracles.

Evaluator	ρ vs R@10	ρ vs NDCG @10	Doc accurac y	Cohen's κ
eRAG	0.790	0.849	0.923	0.331
UMBRELA	0.737	0.593	0.789	0.230
RAGAS-WA	0.744	0.625	—	—

Table 12 — System-level rank agreement and document-level agreement

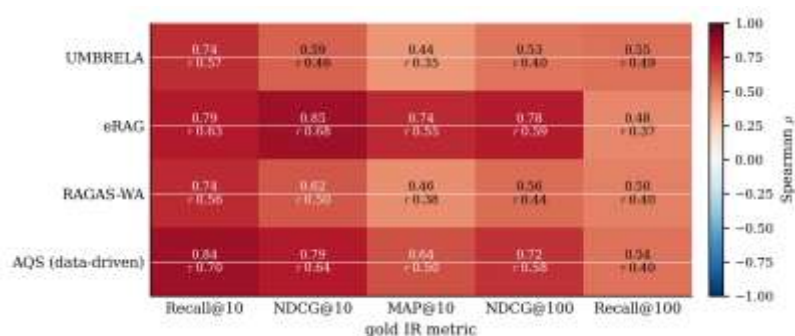


Figure 17 — System-level ranking agreement (19 retrievers)

5.3.2 Where agreement breaks down

The aggregate correlation hides a dependence on the question. As a question's gold set grows, achievable Recall@10 is capped and collapses toward zero (7 of the 48 questions have more than ten gold articles), and only eRAG's score tracks that collapse, UMBRELA's and RAGAS-WA's stay roughly flat (Figure 18). Agreement also varies by stratum: pooled across the 19 systems it is solid within a lexical-alignment stratum but weakest on questions without cross-references (eRAG $\rho \approx 0.09$). A single evaluator score is therefore hard to read in isolation; its reliability depends on the regime the question falls in.

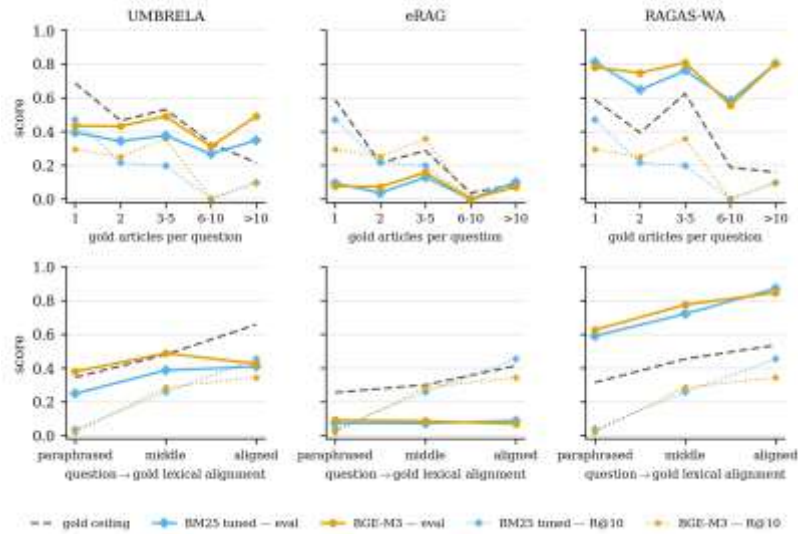


Figure 18 — Regime-dependence of evaluator agreement

5.3.3 Trustworthiness and the gold-ceiling inversion

A high rank correlation says the evaluators *order* systems well; the gold-ceiling diagnostic asks whether their *absolute* scores mean anything. Applied to a gold-only retrieval, each evaluator should score the oracle at least as high as any real retriever. Two of the three do not (Table 13 and Figure 19, left): UMBRELA and RAGAS-WA rate the best real retriever *above* the gold-only oracle (headroom -0.06 and -0.32), so they do not recognise the ground truth as the correct answer and reward surface plausibility over correctness. eRAG is the only evaluator without this inversion, its oracle scores correctly above the best retriever (headroom $+0.11$). The pattern holds across difficulty (Figure 19, right): eRAG shows positive headroom on easy, single-gold questions that shrinks as difficulty rises, whereas RAGAS-WA over-credits at every difficulty (-0.19 to -0.65) and UMBRELA over-credits only beyond ten gold articles (-0.28).

Evaluator	Gold ceiling	Best retriever	Subset mean	Headroom	Inversion
eRAG	0.187	0.081	0.053	+0.105	No
UMBRELA	0.380	0.442	0.324	-0.062	Yes
RAGAS-WA	0.449	0.764	0.605	-0.316	Yes

Table 13 — Gold-ceiling diagnostic

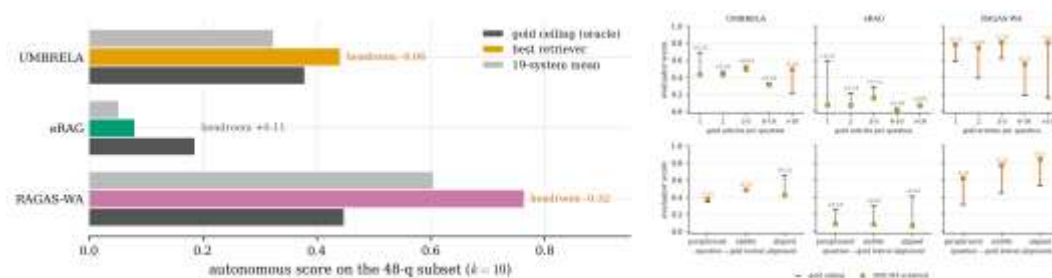


Figure 19 — Gold-ceiling diagnostic and headroom by question difficulty

Taken together, the comparison points to a single recommendation: **eRAG is the most reliable reference-free signal on this corpus**. It best matches the supervised ranking ($\rho \approx 0.79\text{--}0.85$) and is the only evaluator whose absolute scores are trustworthy in the gold-ceiling sense. UMBRELA and RAGAS-WA remain usable as relative system rankers but should not be read as measures of correctness.

Across the three studies, then, Chapter 5 has delivered the empirical core: the matched-pool agentic result for RQ1, the metadata-versus-navigation result for RQ2, and the evaluator comparison for RQ3. Chapter 6 interprets what they mean for the design of legal retrieval systems and bounds their generalisability.

6 Discussion

Chapter 5 reported what the three controlled comparisons yield; this chapter interprets them, answering each research question (§6.1–§6.3), drawing out the implications for legal-RAG system design (§6.4), and bounding where the conclusions hold (§6.5). No new results are introduced.

6.1 Answer to RQ1

RQ1 asked how lexical, dense, hybrid, and agentic retrieval compare on regulatory text. The answer is a clear, significant ordering, sparse → dense → hybrid → LLM scorer, each tier fixing the previous one's failure (dense recovers the paraphrased queries sparse misses, fusion captures both regimes, the scorer lifts the top of the list), with one pointed qualification: the gain at the top comes from introducing the language model *as a scorer*, not from the agentic machinery around it. At a matched candidate-pool size neither CRAG nor ReAct improves significantly on a single-pass relevance judgement; as an ablation, the scorer step is worth +0.043 Recall@10 and the agentic step essentially nothing. On this corpus the value of "agentic RAG" is the language model in the loop, not the loop.

Two practical riders follow. First-stage strength dominates, the same scorer over a sparse first stage falls well short of its hybrid result, so effort is better spent on a strong first stage than on an agentic layer over a weak one. And the agentic loops are not free: they spend extra language-model calls per query for no measurable benefit, leaving the single-pass scorer the most cost-effective operating point.

6.2 Answer to RQ2

RQ2 asked whether exploiting document structure improves retrieval, and at what cost. The answer separates by *where* structure is used. Used lightly, at indexing time, it helps: enriching structural units with an LLM-generated summary improves ranking quality at no query-time cost, and that summary is the only enrichment that moves the metric, the query-time boost/filter being inert. Used heavily, at retrieval time, it does not pay: a vectorless LLM navigator descending the hierarchy is far slower and far less accurate than the vector retrievers, however well the tree is built.

The deeper point is that the structure-aware advantage is one of *ranking*, *not coverage*. The structure-agnostic baseline already retrieves nearly every relevant article somewhere in its deep pool, its weakness is ranking depth, not missing content, and metadata enrichment improves exactly that. The navigation follow-up confirms it from the other side: even after rebuilding the tree, adding an agentic navigator, restoring reach with a corrective loop, and re-ranking with an embedding model, the bottleneck was never the index (only two of nearly three thousand gold articles are genuinely unreachable) but the ranking of what is reached, and navigation becomes competitive with vectors only on the hardest, least lexical codes, at the heaviest query-time cost. The recommendation: add structure to rank and clean context, not to widen coverage, and keep it at indexing time.

6.3 Answer to RQ3

RQ3 asked whether a reference-free pipeline can rank retrieval systems without ground truth. The answer is a qualified yes — qualified by *which evaluator* and *which use*. For system ranking, one evaluator does substitute for supervised evaluation: eRAG's ordering agrees with the supervised ranking at $\rho \approx 0.79$ –0.85, enough to be useful without gold annotations. But it holds for eRAG specifically: UMBRELA and RAGAS-WA rank acceptably yet show a gold-ceiling inversion, rating real retrievers *above* a gold-only oracle, rewarding documents that look as if they could answer over those that do, so their absolute scores cannot be read as correctness. eRAG is the only evaluator without the inversion.

The broader lesson is *how* to use such evaluators. None is a reliable per-document relevance judge, all agree poorly with human labels at the document level, so their value is at the system-ranking layer, in aggregate. A practitioner without gold data can use eRAG to rank candidate configurations and expect the order to track supervised quality, but should treat its absolute scores, and any per-query reading, with caution.

6.4 Cross-cutting implications for legal-RAG system design

Read together, the three studies sketch a coherent recipe for building *and* evaluating retrieval in a regulatory RAG system, the practical question behind the Deloitte collaboration (§1.1). On the retrieval side: a strong hybrid first stage, a single-pass LLM relevance scorer, and indexed units carrying lightweight index-time metadata enrichment. Each piece is earned by a different study, fusion is the best non-LLM retriever and bounds everything above it (§6.1), the scorer is the one language-model addition worth its cost, and index-time enrichment improves ranking at no query-

time cost (§6.2). Conspicuously absent are the agentic loops and the vectorless navigator: both add heavy query-time cost for no retrieval benefit here, and that budget is better spent on the first stage.

On the evaluation side, where gold annotations are usually unavailable in deployment, eRAG supplies a reference-free signal that ranks configurations in agreement with supervised quality (§6.3), enough to tune and monitor the pipeline without continuous expert annotation, provided it is read as a relative ranker, not an absolute score.

The thread through all three studies is that the expensive, elaborate option is rarely the one that pays: a single scoring pass beats agentic loops, index-time enrichment beats query-time navigation, and a cheap abstention-based evaluator beats more elaborate LLM judges. On regulatory retrieval, the recurring winner is the modest mechanism at the right layer, a strong first stage, one scoring pass, light enrichment.

6.5 Limitations and generalisation

Three classes of limitation bound these conclusions.

Boundary conditions on the matched-pool finding. The agentic-value result is specific to its setting, and at least four conditions could reverse it: the pool was held at twenty for a clean match (a deeper pool might give the loops room to surface what a single pass cannot); BSARD questions are short and single-intent (multi-hop or compositional queries, for which ReAct was designed, might exercise the loop more favourably); the first stages are zero-shot (fine-tuning could shift the balance); and the backbone is an 8-billion-parameter model (a larger one might change the calculus). The finding holds for short, single-intent queries over a closed corpus with a zero-shot first stage and a small open model, common, but not universal.

Benchmark-specific limitations. BSARD's high answer multiplicity caps Recall@10 well below one for many questions, so R@10 understates quality on multi-article questions; the lexical-alignment strata are a BM25-score proxy, not hand-annotated; the ground truth is a snapshot that drifts from current law; and the 222-question test split limits fine-grained stratified power, so per-stratum differences are directional rather than confirmed. RQ2 adds an arm-to-arm pool-depth asymmetry (Recall@100 favours the deeper baseline), and RQ3 rests on a 48-question subset scored by a single judge model, applied to first-stage retrievers only.

Generalisation beyond BSARD. The findings most likely to transfer are the structural ones, since they follow from corpus properties rather than this benchmark: the lexical/semantic complementarity that makes hybrid fusion win, and the

substitution of an LLM scorer for a cross-encoder, should hold on other closed, terminologically-dense regulatory corpora, tax codes, building codes, other national statutes. Least transferable are the precise stratum locations and the exact agentic-value margin, which depend on the question-versus-corpus language distribution and model scale. All experiments are on French text; the Dutch BSARD and other languages are untested.

These limitations motivate the future work of the concluding chapter.

7 Conclusion and Future Work

This thesis investigated how to enhance the performance and quality of context retrieval for regulatory document corpora, through three controlled studies on the BSARD benchmark, each isolating a single design axis so its effect could be measured cleanly.

7.1 Summary of contributions

The retrieval comparison (RQ1) established a significant ladder from sparse to dense to hybrid to a language-model scorer, and isolated the central result: at a matched candidate-pool size, agentic correction and reasoning loops do not significantly improve on a single-pass LLM relevance scorer. The retrieval value of agentic retrieval, on this corpus, is the value of the language model, not the loop; the closed-domain adaptation of CRAG, intra-corpus query rewriting in place of web search, adds significant value over the non-LLM baseline but does not exceed the non-agentic scorer.

The structure-aware comparison (RQ2) showed, through one common scoring path, that lightweight index-time metadata enrichment improves ranking quality at no query-time cost while vectorless navigation is dominated; a deep-tree follow-up then demonstrated constructively that navigation's deficit is one of *ranking*, not coverage, restoring reach and re-ranking the reached pool with an embedding model makes it competitive with vector retrieval on the hardest codes, but only there and at the heaviest query-time cost. The evaluation study (RQ3) identified eRAG as the one reference-free evaluator whose system ranking agrees with supervised quality and whose absolute scores are not inverted, while surfacing a gold-ceiling inversion in the others that bounds how their scores may be read.

Across all three, the recurring lesson is that a modest mechanism at the right layer outperforms more elaborate machinery, a strong hybrid first stage with a single scoring pass, lightweight structural enrichment at indexing time, and a well-chosen reference-free metric for monitoring. In support, the thesis also produced a custom pipeline for extracting and matching Belgian statutory articles from source PDFs, with a quantification of the drift between the benchmark's ground truth and current law.

7.2 Future work

Retrieval. The matched-pool finding holds under specific conditions, and four follow-ups would test its boundaries: deeper candidate pools, multi-hop or compositional query subsets, a fine-tuned dense first stage, and larger language-model backbones.

Structure-aware retrieval. Richer, non-deterministic query-time metadata signals, and combining the whole-article unit's coverage with the summary-node unit's ranking quality, are natural extensions of the metadata approach.

Evaluation. Extending the reference-free panel from the 19 first-stage retrievers to the hybrid and agentic systems would test whether the eRAG recommendation and the gold-ceiling inversion persist on downstream pipelines that re-rank rather than retrieve.

Deployment. Applying the selected pipeline to the PDF-extracted corpus and to the Dutch variant of the benchmark would test generalisation from canonical text to noisier, drifted text and to another language.

7.3 Closing reflection

Returning to the primary research question, how the performance and quality of context retrieval can be enhanced for regulatory corpora, the thesis answers it consistently across retrieval, structure, and evaluation: enhancement comes not from the most elaborate mechanism but from the right one at the right layer. On regulatory text, where retrieval is the bottleneck of the whole pipeline, knowing which sophistication pays for itself is as valuable as the sophistication itself.

Bibliography

- Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR*.
- Bruch, S., Gai, S., & Ingber, A. (2023). An Analysis of Fusion Functions for Hybrid Retrieval. *ACM TOIS*, 42(1).
- Chalkidis, I., Fergadiotis, M., Malakasiotis, P., & Androutsopoulos, I. (2019). Large-Scale Multi-Label Text Classification on EU Legislation. *ACL*.
- Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., & Androutsopoulos, I. (2020). LEGAL-BERT: The Muppets straight out of Law School. *Findings of EMNLP*.
- Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024a). *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. arXiv preprint.
- Chen, T. (2024b). *Dense X Retrieval: What Retrieval Granularity Should We Use? *EMNLP**. arXiv preprint.
- Cormack, G., Clarke, C., & Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR*.
- Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated Evaluation of Retrieval Augmented Generation. *EACL (System Demonstrations)*.
- Gao, L., Ma, X., Lin, J., & Callan, J. (2023a). Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). *ACL*.
- Gao, Y., Xiong, Y., & Gao, X. (2023b). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv preprint.
- Goebel, R., Kano, Y., Kim, M.-Y., Rabelo, J., Satoh, K., & Yoshioka, M. (2024). Overview and Discussion of the Competition on Legal Information Extraction/Entailment (COLIEE) 2023. *The Review of Socionetwork Strategies*, 18(1), 27–47.
- Grattafiori, A. (2024). *The Llama 3 Herd of Models*. arXiv preprint.
- Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M.-W. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. *ICML*.

- Izacard, G., Caron, M., & Hosseini, L. (2022). Unsupervised Dense Information Retrieval with Contrastive Learning (Contriever). *TMLR*.
- Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs (FAISS). *IEEE Transactions on Big Data*, 7(3), 535–547.
- Karpukhin, V., Oğuz, B., & Min, S. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP*.
- Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. *SIGIR*.
- Lewis, P., Perez, E., & Piktus, A. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.
- Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., & Zhu, C. (2023). G-Eval: NLG Evaluation Using GPT-4 with Better Human Alignment. *EMNLP*.
- Louis, A., & Spanakis, G. (2022). A Statutory Article Retrieval Dataset in French (BSARD). *ACL*.
- Louis, A., van Dijck, G., & Spanakis, G. (2024). Interpretable Long-Form Legal Question Answering with Retrieval-Augmented Large Language Models. *AAAI*.
- Ma, X., Gong, Y., He, P., Zhao, H., & Duan, N. (2023). Query Rewriting in Retrieval-Augmented Large Language Models. *EMNLP*.
- Martin, L., Muller, B., & Ortiz Suárez, P. (2020). CamemBERT: a Tasty French Language Model. *ACL*.
- Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive Text Embedding Benchmark. *EACL*.
- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Salemi, A., & Zamani, H. (2024). Evaluating Retrieval Quality in Retrieval-Augmented Generation (eRAG). *SIGIR*.
- Sarathi, P., Abdullah, S., & Tuli, A. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *ICLR*.
- Sun, W., Yan, L., & Ma, X. (2023). Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents (RankGPT). *EMNLP*.

- Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models. *NeurIPS (Datasets & Benchmarks)*.
- Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2023). Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions (IRCoT). *ACL*.
- Upadhyay, S., Pradeep, R., Thakur, N., Craswell, N., & Lin, J. (2024). *UMBRELA: Umbrela is the (Open-Source Reproduction of the) Bing RElevance Assessor*. arXiv preprint.
- Vectify AI. (2025). *PageIndex: Document Index for Vectorless, Reasoning-based RAG [Software]*. Retrieved from <https://github.com/VectifyAI/PageIndex>
- Wang, L., Yang, N., & Huang, X. (2024). *Multilingual E5 Text Embeddings: A Technical Report*. arXiv preprint.
- Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). *Corrective Retrieval Augmented Generation (CRAG)*. arXiv preprint.
- Yao, S., Zhao, J., & Yu, D. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*.
- Zheng, L., Chiang, W.-L., & Sheng, Y. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS (Datasets & Benchmarks)*.

Code of Conduct

GenAI code of conduct and transparency statement for students 2025-2026.

Code of conduct and transparency statement on the use of GenAI for KU Leuven-students (academic year 2025-2026)

Generative AI (GenAI) assistance tools can be used to generate various types of content, including text, images, code, video, music, or combinations thereof. Common examples of such tools include ChatGPT, Google Gemini, Microsoft Copilot, Midjourney, Claude.ai, Perplexity.ai, and DALL·E, among others.

This code of conduct is a tool that helps students to be transparent about the use of GenAI and fits within the university's principles on academic integrity.

Important guidelines and remarks

- **Sensitive or personal data:** Some GenAI tools protect your input and sensitive or personal data better than others. There is often no transparency on what the owners of the AI applications do with the data entered. Therefore, **do not enter sensitive or personal data in free GenAI tools**. More info about what sensitive and personal data are, is described in the [three confidentiality levels of the KU Leuven data classification model](#) (non-confidential, confidential, strictly confidential). For confidential data you should preferably use **M365 Copilot Chat with your KU Leuven account**. If you use another GenAI tool, you must be **absolutely certain** it does **not store or reuse the data you enter**. Try to avoid entering these data. Do so only if strictly necessary, and only to the extent required. For personal data, work with anonymous or pseudonymized data. Additionally, in case of strictly confidential data this should first be discussed with the teaching staff of the course or your thesis supervisor.
- **Copyrighted materials:** For lawfully obtained copyrighted material you should preferably use **M365 Copilot Chat with your KU Leuven account**. If you use another GenAI tool, you must be **absolutely certain** it does **not store or reuse the data you enter**.
- GenAI assistance may not be used for data or topics covered by a [Non-Disclosure Agreement \(NDA\)](#). Even in the absence of an NDA, certain information may still need to be treated as confidential—for example, due to regulatory requirements or the risk of significant harm to the university if disclosed. In case of doubt, check with your teaching staff or supervisor.
- If your master's thesis is under [embargo](#), you should first discuss with your supervisor whether the use of GenAI is permitted.
- Before using a GenAI tool, always consider whether its use is responsible, including from a [sustainability perspective](#) (e.g. when using GenAI as a search engine, language assistant,...).
- **Take a scientific and critical attitude** when interacting with GenAI assistance and interpreting its output, that may not always be correct.
- As a student you are responsible for complying with Article 84 of the Regulations on Education and Examinations: your report or thesis should reflect

your own knowledge, understanding and skills. Be aware that plagiarism rules also apply to (work that is the result of) the use of GenAI assistance tools.

Exam Regulations Article 84: “Every conduct individual students display with which they (partially) inhibit or attempt to inhibit a correct judgement of their own knowledge, understanding and/or skills or those of other students, is considered an irregularity which may result in a suitable penalty. A special type of irregularity is plagiarism, i.e. copying the work (ideas, texts, structures, designs, images, plans, codes , ...) of others or prior personal work in an exact or slightly modified way without adequately acknowledging the sources. Every possession of prohibited resources during an examination (see article 65) is considered an irregularity.”

- In order to maintain academic integrity and avoid plagiarism, **more information about being transparent on the use of GenAI assistance and about correctly citing and referencing GenAI** can be found on this website [for students \(Dutch/English\)](#).
- **Additional reading: KU Leuven guidelines on responsible use of Generative AI tools, and other information** ([Dutch/English](#))

A few final words

If you are uncertain whether or not you should declare your use of GenAI tools, we suggest that you discuss this with your instructor or supervisor. It is always safer to declare GenAI use, even when it is not strictly required. However, declaring GenAI use does not entail that its use is allowed; the right column in the table below provides more detailed instructions in this regard (code of conduct).

Moreover, advanced AI tools are evolving rapidly, and their capabilities have expanded significantly in a short period of time. As a result, we do not yet have all the answers about their responsible use. Finally, it is important to follow-up on the most recent evolutions in AI technologies, to have an open mind but also to be a bit cautious, to communicate with instructors, teaching assistants, supervisors and peers, to be as transparent as we can, and to learn together as we move along.

Student name:

Student number: r1074160

Please indicate with "X" whether it relates to a course assignment or to the Bachelor's or Master's thesis:

☐ This form is related to a **course assignment**.

Course name:

Course number:

This form is related to **my** ☐ Bachelor's or ☒ Master's thesis.

Title Bachelor's or Master's thesis:

Enhancing Performance and Quality of Context Retrieval in
Retrieval-Augmented Generation (RAG) Systems

Supervisor: Dr.Tim Van de Cruys

Daily supervisor: Deloitte – Yuanyuan Li
Deloitte – Florian Geron
Deloitte – Grzegorz Przesniak

Please indicate with "X":

☐ I did not use any GenAI assistance tool.

X I did use GenAI Assistance. In this case **specify which ones** (e.g. ChatGPT, M365 Copilot,...):

GenAI assistance used as/for:	Name of the GenAI tool(s) used. If helpful, also describe in which way you were using GenAI related to what is specified as code of conduct.	Code of conduct:
		For each of the categories below, always take into account the important guidelines and remarks mentioned above (e.g. copyrighted data, sensitive or personal data,...).
As a language assistant for reviewing or improving texts I wrote myself	Claude	This use is similar to using spelling and grammar check tools. In general, you do not have to refer to such kind of GenAI use in the text. However, be careful: - When using GenAI tools on texts you did not write yourself to improve the text, you have to <u>refer</u> to the original source or author , otherwise you are committing <u>plagiarism</u> and thus an irregularity.
As a paraphrasing tool	Claude	This use is allowed except when it is prohibited by the teacher or the program of study. You may paraphrase your own text or texts by an author other than yourself and take inspiration from what a GenAI tool or another tool suggests (unless it is not allowed). In general, you do not have to

		refer to such kind of GenAI use or other paraphrasing tools in the text. However, be careful: - If it entails text by an author other than yourself, you are not allowed to include that paraphrased text without <u>reference</u> to the original source or author. Without such reference, you would be committing <u>plagiarism</u> and thus an irregularity.
For translation aid to improve texts I wrote myself or to better understand text from others	Claude	This use is allowed except when it is prohibited by the teacher or the program of study. It is similar to using translation tools (Google translate, DeepL, ...). In general, you do not have to refer to such kind of GenAI use in the text. However, be careful: - You are not allowed to include that translated text without <u>reference</u> to the original source or author. Without such reference, you would be committing <u>plagiarism</u> and thus an irregularity. - Always check the translated text for correctness and meaning.
As a search engine to get information on a topic or	Claude	This use is similar to e.g. a Google search or checking Wikipedia. If you write your own text based on this information, you do not have to

to search for existing research on the topic		refer to the use of GenAI in the text. You only have to refer to the existing research and references you have checked and used (without such references you would be committing plagiarism and thus committing an irregularity). However, be careful: - Be aware that the output of the GenAI tool cannot be guaranteed as a 100% reliable source of information. The output may not be entirely correct and/or be limited due to the databases it uses. Moreover, knowledge evolves and may change over time; therefore, the database of the GenAI tool may not be up to date. Therefore, verify the information and do not just copy-paste it as you should understand and critically process everything you are writing.
For literature search	Claude	This use is comparable to e.g. a Google Scholar search. You do not have to refer to such kind of GenAI use; you only have to refer to the literature references you have checked and used (without such references you would be committing plagiarism and thus committing an irregularity).. However, be careful: - Be aware that the search output is restricted to the database the GenAI tool is

Code of Conduct

		built on. After this initial search, look for scientific sources and conduct your own analysis of the source documents. Interpret, analyse and process the information you obtained; verify it and do not just copy-paste it as you should understand and critically process everything you are writing. - Be aware that some GenAI tools may output <u>no or wrong references</u>. As a student you are responsible for further checking and verifying the absence or correctness of references; do not just copy-paste it.
For generating programming code	Claude	Use of GenAI for coding is allowed except when it is prohibited by the teacher or the program of study. If used for coding, correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI.
For generating new (research) ideas	Claude	This use of GenAI is allowed except when it is prohibited by the teacher or the program of study. Correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI. Be careful: - Further verify in this case whether the idea is novel or not. It is likely that it is

		related to existing work. If so, that existing work should be correctly referenced in the text (without such reference you would be committing plagiarism and thus committing an irregularity).
For generating synthetic data	Claude	Use of GenAI for generating synthetic data is allowed, provided that it is methodologically and ethically justifiable, except when it is prohibited by the teacher or the program of study. Always correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparant about the use of GenAI. Be careful: - Always carefully evaluate the generated synthetic data for quality and possible bias since the output is highly dependent on the quality of the data on which the models are trained.
For generating blocks of text (other than the allowed use without referencing mentioned above)	Claude	According to Article 84 of the Regulations on Education and Examinations your text should allow to correctly and properly assess your own knowledge, understanding and skills. Therefore, inserting blocks of text without quotes and a reference to GenAI assistance in your work is not allowed. Be careful:

Code of Conduct

		- If it is really needed to insert a block of text from a GenAI tool, for instance because of the nature of your assignment, mention it as a citation by using quotes and correctly mention the use of GenAI assistance in accordance with the instructions page on being transparant about the use of GenAI. - However, in general, such GenAI use should be kept to an absolute minimum; you should always check the original sources.
For generating visuals, video or audio	Claude	Use of GenAI for generating visuals, audio or video is allowed except when it is prohibited by the teacher or the program of study. Be careful: - If used, refer to GenAI for visuals according to the usual referencing style, following the instructions on the page on referencing GenAI. - If you work with existing visuals, audio or video, that existing work should be correctly referenced in the text (without such reference you would be committing plagiarism and thus an irregularity). - Explain the usage in the methods section (if there is one) and optionally attach (or

		link to) the prompts with the full output (or history).
Other use (specify here; this may also include a combination of types of use mentioned above):		To make sure other use of GenAI is allowed within the course or thesis, and if so, the conditions that may apply, contact the teaching staff of the course or the supervisor of the thesis beforehand and explain the intended GenAI use. Also inform the programme director. Motivate how you would comply with Article 84 of the exam regulations. Explain the use and the added value of the AI tool you consider to use and how it is in accordance with the assignment or thesis and the KU Leuven guidelines on responsible use of Generative AI tools. Depending on the kind of GenAI use, it may be needed to properly reference it in the text, in accordance with the instructions page on being transparant about the use of GenAI.